

Digitale Systeme und Computersysteme

Kompetenzbereich Computerarchitekturen



<https://monster6502.com>

Auszug Lehrplan

Die Schülerinnen und Schüler können im Bereich Computerarchitekturen...

... im IV. Jahrgang, 7. Semester

die Funktionsweise von I/O-Komponenten erklären.

... im IV. Jahrgang, 8. Semester

das Hardware-Software-Interface von Standardschnittstellen erklären.

Lehrstoff:

- Peripheriekomponenten

Dieses Skript wurde von **Michael Kupfner** für das Unterrichtsfach **DIC – Digitale Systeme und Computersysteme, 4AHEL, Schuljahr 2025/26** an der HTBLVA Anichstraße erstellt. Eine Weitergabe oder Vervielfältigung außerhalb dieses Unterrichts ist nicht gestattet.

1 I/O-Pin-Komponente

Ein-/Ausgabekomponenten (engl. *I/O – Input/Output*) sind als Bindeglied zwischen einem Mikroprozessorsystem und seiner Umwelt.

Beim in der HTBLVA Anichstraße eingesetzten **PSoC5LP** unterscheidet man drei Arten von I/O-Anschlüssen:

- **GPIO** (*General Purpose I/O*)
- **SIO** (*Special I/O*)
- **USBIO** (*für USB*)

Auszug aus dem PSoC5LP Architecture Technical Reference Manual:

"All I/O pins are available for use as digital inputs and outputs for both the CPU and digital peripherals. [...]. All GPIO pins can be used for analog input, CapSense®, and LCD segment drive, while SIO pins are used for voltages in excess of V_{dda} and for programmable output voltages and input thresholds. [...]"

Merke: I/O-Pins

GPIO: universell verwendbare digitale **und analoge** Ein- und Ausgangsanschlüsse (z. B. LED, Taster, ADC-Eingang).

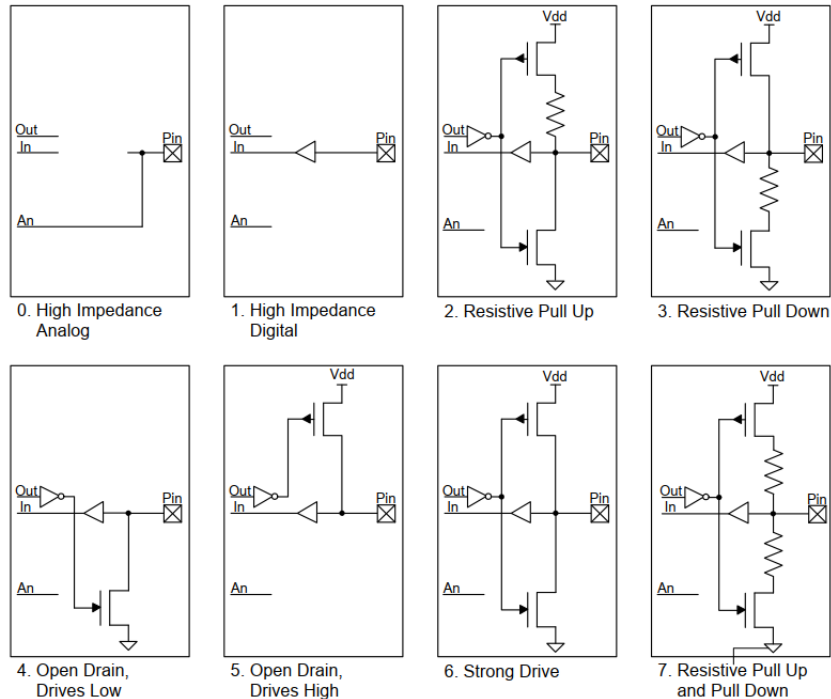
SIO: höhere Strombelastbarkeit, 5 V-tolerant, programmierbare Pegel, **geeignet für Schnittstellen** – aber **ohne Analogfunktion**.

USBIO: spezielle Pins für die USB-Schnittstelle.

1.1 Drive-Modes

Jeder GPIO und SIO Pin kann in einem von acht Betriebsmodi (*Drive Modes*) betrieben werden:

- **High Impedance Analog:** Grundbetriebsart (*factory drive mode default*), Pin elektrisch abgetrennt → keine digitale Funktion.
- **High Impedance Digital:** empfohlen für digitale Eingänge.
- **Resistive Pull-Up / Pull-Down:** interner Widerstand (~5.6 kΩ typisch) gegen VDD bzw. GND.
- **Open Drain (Drives Low / Drives High):** Pin zieht nur aktiv nach GND oder VDD, sonst hochohmig → ideal für Bus-Systeme (z. B. I²C).
- **Strong Drive:** „klassischer“ digitaler Ausgang, treibt aktiv High und Low.
- **Resistive Pull-Up and Pull-Down:** beide Widerstände aktiv, keine niederohmige Ansteuerung.



The 'Out' connection is driven from either the Digital System (when the Digital Output terminal is connected) or the Data Register (when HW connection is disabled).
 The 'In' connection drives the Pin State register, and the Digital System if the Digital Input terminal is enabled and connected.
 The 'An' connection connects to the Analog System.

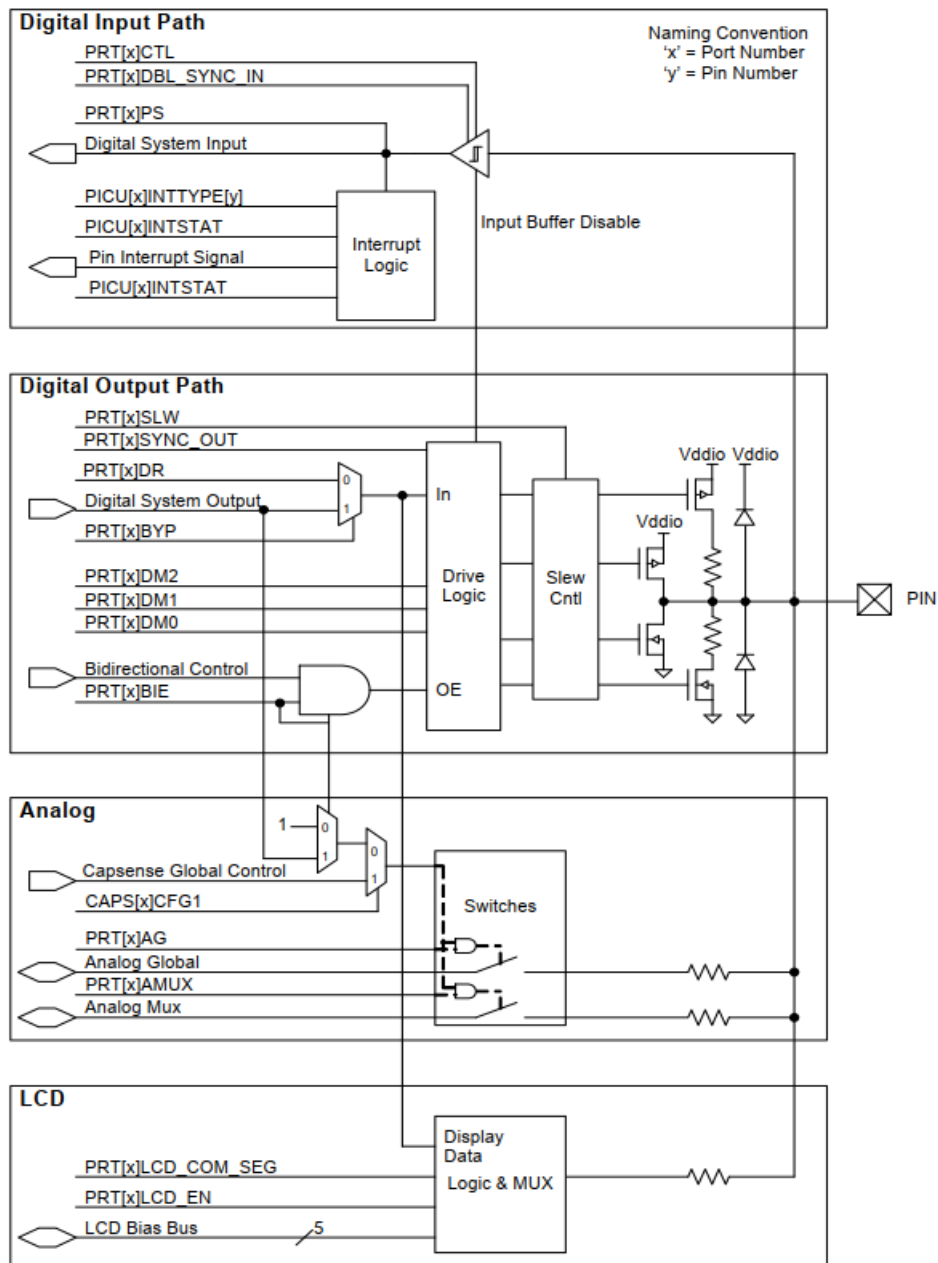
Quelle: PSoC5LP Architecture TRM, Document No. 001-78426 Rev. *C



KI-Aufgabe: Taster mit Pull-Up oder Pull-Down?

Externe Taster können sowohl mit einem Pull-Up-Widerstand ($\rightarrow$ bei Betätigung = log. 0) als auch mit einem Pull-Down-Widerstand ($\rightarrow$ bei Betätigung = log. 1) an einen Pin angeschlossen werden. In der Praxis findet man jedoch viel häufiger die **Pull-Up-Variante** (Taster gegen Masse). Befrage eine KI nach den Gründen dafür und notiere die wichtigsten Argumente. Erkläre deinem Banknachbarn, welche Argumente du am überzeugendsten findest – und hört euch gegenseitig eure Antworten an.

Die folgende Darstellung zeigt, wie komplex ein einzelner GPIO-Anschluss intern aufgebaut ist:



Quelle: PSoC5LP Architecture TRM, Document No. 001-78426 Rev. *C

1.2 Pin-Konfigurationen

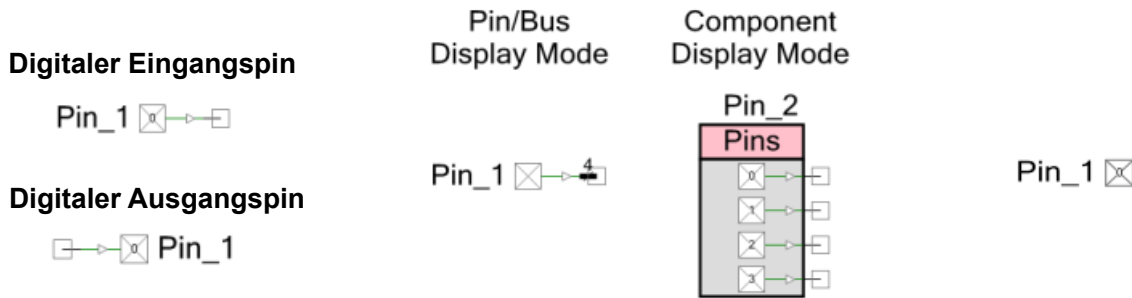
Die Pin-Konfiguration erfolgt beim PSoC komfortabel über die **Pins-Komponente** in PSoC Creator.

- Grafische Auswahl der Betriebsart (*Drive Mode, Slew Rate, ...*)
- Zuweisung zu einem physischen Port-Anschluss
- Zugriff per API während der Laufzeit möglich

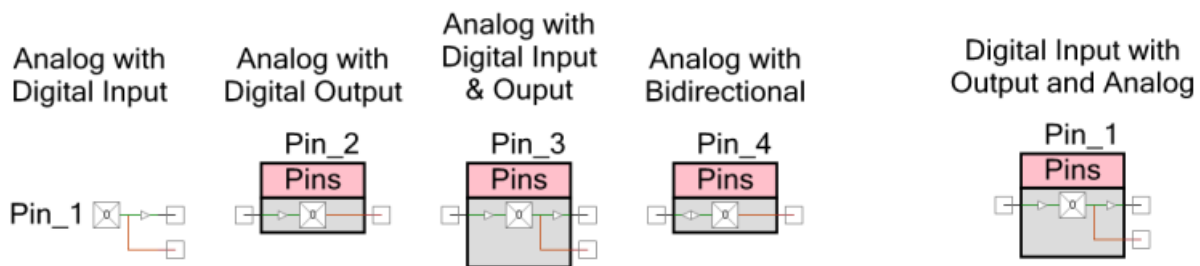
Pin-Grundkonfigurationen

Eine Pin-Komponente kann als einzelner Anschluss oder als Bus konfiguriert werden. Bei Pins, die

ausschließlich vom Prozessor-Subsystem angesteuert werden, fehlt der interne Anschluss an das digitale Subsystem.



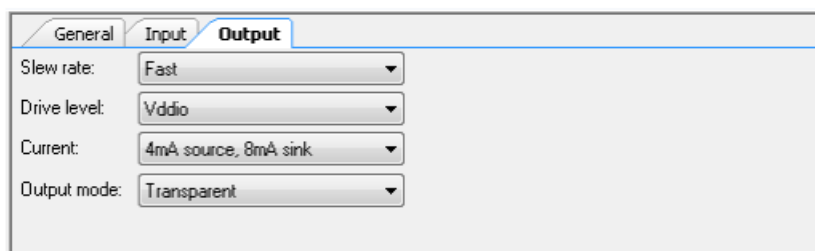
Ein Pin kann gleichzeitig für digitale und analoge Funktionen genutzt werden (z. B. analog messen und digital schalten). Solche Mehrfachkonfigurationen sind technisch möglich, sollten aber nur in speziellen Anwendungen eingesetzt werden, da es leicht zu Konflikten kommt.



Ein Pin kann auch **bidirektional** konfiguriert werden. In diesem Modus arbeitet er sowohl als Eingang als auch als Ausgang. Typisch ist der Einsatz in **seriellen Bussystemen** (z.B. I²C), wo ein Pin je nach Protokollphase Daten senden oder empfangen muss. Darüber hinaus erlaubt die Flexibilität des PSoC5LP auch eine analoge Anbindung an einen digitalen, bidirektionalen Pin – dies ist jedoch ein Spezialfall.



Bei digital konfigurierten Pins (Output oder Bidirectional) können zusätzliche **Ausgangsparameter** eingestellt werden. Diese beeinflussen maßgeblich das elektrische Verhalten und damit sowohl die Signalqualität als auch die EMV-Eigenschaften eines Systems.



Wichtige Parameter:

- **Slew Rate:** bestimmt die Flankensteilheit (*steile Flanken = kurze Übergangszeit, aber mehr hochfrequente Spektralanteile*).

- Niedrige Slew Rate: weniger hochfrequente Spektralanteile, aber längere Übergangszeit
- Hohe Slew Rate: kurze Übergangszeit, aber eventuell EMV-kritisch.
- **Drive Level:** legt die Ausgangsspannung fest.
 - Standard: VDDIO
 - SIO-Pins: erlauben eine von VDDIO abweichende, einstellbare Ausgangsspannung.
- **Drive Current:** maximaler Strom, den ein Pin liefern oder aufnehmen kann.
 - Standard: 4 mA Source (Quelle; liefernd), 8 mA Sink (Senke; aufnehmend)
 - SIO-Pins: bis zu 25 mA Senke möglich
- **Output Synchronization:** reduziert Laufzeitunterschiede (*Signal-Skew*) zwischen mehreren Pins bei schnellen Signalen.
 - Standard: *Transparent* (keine Synchronisation, Signal wird direkt ausgegeben).
 - Option: *Single-Sync* (das Ausgangssignal wird mit dem Systemtakt synchronisiert, dadurch geringerer Laufzeitunterschied).

Das CPU-Subsystem hat über das **API** (*Application Programming Interface*) Zugriff auf die Pin-Register und kann auf diese lesend und schreibend zugreifen. Damit ist es auch möglich, während der Laufzeit die Pineinstellungen zu ändern. Das Pin-API stellt -auszugsweise- folgende Aufrufe zur Verfügung:

```
uint8 Pin_Read(void);           // liest Pinstatus
void Pin_Write(uint8 value);    // schreibt Wert an Pin
void Pin_SetDriveMode(uint8 mode); // setzt Betriebsmodus
```

2 Interrupts

Viele Ereignisse im System passieren **asynchron** zur Programmabarbeitung: Ein Taster wird gedrückt, ein Timer läuft ab, eine Nachricht wird empfangen. Statt solche Ereignisse **ständig** zyklisch **abzufragen (Pollen)**, kann bei Auftreten eines solchen Ereignisses die CPU in ihrer Programmabarbeitung unterbrochen werden – über einen **Interrupt**.

2.1 Ausnahmen (Exceptions)

Eine **Ausnahme** (engl. *Exception*) kann die reguläre **Programmausführung** unterbrechen. Man unterscheidet zwischen:

- **Synchron (Exceptions/Traps):** entstehen **durch die gerade laufende Instruktion**, z. B. **Division durch 0**, **Adressfehler**, **arithmetischer Überlauf** oder **gezielte Auslösung durch Software (SVC/Trap)**.
- **Asynchron (Interrupts, IRQ):** entstehen **außerhalb** der aktuellen Instruktion, z. B. durch **Peripherie** (GPIO-Ereignis, Timer-Überlauf, UART-Empfang), **Versorgungseinbruch** oder **externe Signale**.

Merke: Exception vs. Interrupt

Exception (synchron): Ursache = aktuelle Instruktion.

Interrupt (asynchron): Ursache = äußeres Ereignis.

Tritt eine solche Ausnahmesituation auf, verzweigt der Prozessor in eine spezielle Routine (*Exception Handler* bzw. *Interrupt Service Routine, ISR*). Manche μ C-Architekturen erlauben die **gezielte Auslösung von Traps per Befehl**. Dadurch kann in einen speziellen / abgesicherten Systemmodus gewechselt werden (*Supervisor / Protected Mode*). Typische Anwendungen: Debugging, Betriebssystem-Routinen.

Asynchrone Unterbrechungen haben **nichts mit der gerade ausgeführten Instruktion zu tun**. Sie treten entweder unvorhersehbar auf (z. B. Einbruch der Versorgungsspannung) oder werden typischerweise durch **Peripherie** verursacht. Dafür stehen ein oder mehrere Interrupt-Eingänge zur Verfügung. Tritt ein Interrupt-Ereignis auf, wird in die diesem Interrupt zugeordnete **Interrupt-Service-Routine (ISR)** verzweigt.

Je nach Architektur muss in der ISR entweder zunächst festgestellt werden, **welche Quelle den Interrupt ausgelöst hat**, oder es gibt eine **Vektortabelle** im Speicher. In dieser Tabelle sind die Startadressen (Vektoren) der jeweiligen ISRs abgelegt.

Technischer Ablauf beim Auftreten einer Ausnahme

1. Der Prozessor stoppt die aktuelle Abarbeitung.
2. Programmzähler (PC) und Statusregister (Flags) werden automatisch auf dem **Stack** gesichert.
3. Die CPU springt
 - entweder in eine **zentrale ISR** → diese muss den Auslöser identifizieren,
 - in die dafür **vorgesehene ISR**, die **Startadresse** steht in der **Vektortabelle**.

Jedem Interrupt wird auch eine **Priorität** zugeteilt; ein Interrupt mit einer **höheren Priorität** kann einen Interrupt mit einer **niedrigeren Priorität** unterbrechen – wobei eine *höhere Priorität* nicht zwangsläufig in Form eines größeren Zahlenwertes ausgedrückt wird; bei vielen Architekturen hat die Prioritätsnummer 0 die allerhöchste Priorität.

Der Programmierer ist meist auch in der Lage, festzulegen, welche Interrupts wann zugelassen werden und welche nicht. D.h. je nach Programmablauf kann die Abarbeitung von ISR unterbunden oder eingeschränkt werden.

i Info: Maskierbare vs. Nicht maskierbare Interrupts

IRQ (Interrupt Request):

- kann vom Programmierer freigegeben oder gesperrt werden (*maskierbar*).
- Beispiele: Timer, UART, GPIO.

NMI (Non Maskable Interrupt):

- *nicht* deaktivierbar, da für kritische Ereignisse reserviert.
- Beispiele: Versorgungseinbruch, Watchdog, Hardwarefehler.

Wichtig: NMIs stellen sicher, dass das System auch bei Fehlern oder Notfällen noch reagieren kann.

💡 Merke: Interrupt vs. Polling

Interrupt: Ereignisquellen melden ihre Signale asynchron. Die CPU unterbricht ihre aktuelle Arbeit und springt in eine Interrupt Service Routine (ISR).

Vorteile: geringe Latenz, energieeffizient (Sleep möglich), keine nutzlosen Abfragen

Nachteile: höhere Komplexität (Prioritäten, Nebenläufigkeit), ISR muss kurz sein

Typische Anwendungen: UART-Empfang, Timer, schnelle Impulse, zeitkritische Vorgänge

Polling: Die CPU fragt die Ereignisquellen **periodisch** ab.

Vorteile: einfach, gut nachvollziehbar, deterministisch bei fester Abtastrate

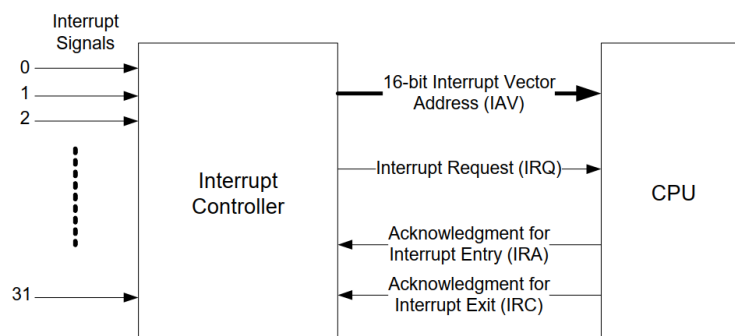
Nachteile: CPU-Zeit wird ständig verbraucht, Latenz = Abfrageintervall, kurze Ereignisse können übersehen werden

Typische Anwendungen: Tasterabfragen, Statusbits, einfache Zustandsautomaten

2.2 Interrupts beim PSoC5LP

Der PSoC5LP besitzt einen leistungsfähigen Interrupt-Controller, der Interruptanforderungen der Peripherie verarbeitet und an die Cortex-CPU meldet. Damit lassen sich Interrupts flexibel verwalten und priorisieren.

Figure 7-1. Interrupt Controller Block Diagram



Quelle: PSoC5LP Architecture TRM, Document No. 001-78426 Rev. *C

- **Interrupt-Eingänge:** bis zu **32 externe Quellen** können Interrupts auslösen.
- **Prioritäten:** administrierbar von **0 (höchste) bis 7 (niedrigste)**. Diese Werte können auch **zur Laufzeit** geändert werden.
- **Triggerarten:** jede Quelle kann auf **Level (Pegel)** oder **Pulse (Flanke)** reagieren.

Zusätzlich kennt der PSoC5LP **15 verschiedene Exceptions** - darunter **nicht-sperrbare Interrupts**, die durch eine **negative Priorität** gekennzeichnet sind (z. B. NMI):

Table 7-3. PSoC 5LP Exceptions

Interrupt Number	Exception Type	Priority	Comments
1	Reset	-3 (highest) not programmable	Reset
2	NMI	-2 not programmable	Non-Maskable Interrupt
3	Hard Fault	-1 not programmable	All fault conditions if the corresponding handler is not enabled
4	MemManage Fault	Programmable	Memory management fault; MPU violation or access to illegal locations
5	Bus Fault	Programmable	Bus error; occurs when AHB interface receives an error response from a bus slave (also called prefetch abort if it is an instruction fetch or data abort if it is a data access)
6	Usage Fault	Programmable	Exceptions due to program error
7	Reserved	NA	--
8	Reserved	NA	--
9	Reserved	NA	--
10	Reserved	NA	--
11	SVCall	Programmable	System Service Call
12	Debug Monitor	Programmable	Debug monitor (watchpoints, breakpoints, external debug request)
13	Reserved	NA	--
14	PendSV	Programmable	Pendable request for system device
15	SYSTICK	Programmable	System Tick Timer

Quelle: PSoC5LP Architecture TRM, Document No. 001-78426 Rev. *C



KI-Aufgabe: Nesting von Interrupts

Der PSoC5LP unterstützt auch das *Nesting* von Interrupts. Bitte eine KI, dir diesen Begriff technisch zu erklären, und finde ein Beispiel aus der Praxis, bei dem Nesting sinnvoll eingesetzt wird. Diskutiere anschließend auch mit der KI, welche Probleme entstehen können, wenn Nesting ohne Einschränkung zugelassen wird.

3 Timer/Counter

Zählen gehört zu einer der wichtigsten Funktionen eines μC . Typische Anwendungen sind z. B. das **Zählen externer Ereignisse** (Frequenzmessung) oder die **Erzeugung von Zeitverzögerungen** (z. B. 1 s für eine Uhrfunktion). Zeitverzögerungen „*per for-Schleife*“ sind ungenau und blockieren die CPU, da die Ausführungszeit von Compiler, Taktfrequenz und Interrupts abhängt. **Zählen** und **Zeit messen/erzeugen** sind dagegen klassische **Hardware-Aufgaben**: Dafür besitzen Mikrocontroller dedizierte **Timer/Counter-Einheiten**, die -einmal initialisiert- **eigenständig** arbeiten, **Interrupts/Ereignisse** erzeugen oder **auf externe Signale reagieren**.



Merke: Im Prinzip besteht eine Timer/Counter-Komponente aus einem **hardwarebasierten Zähler**.

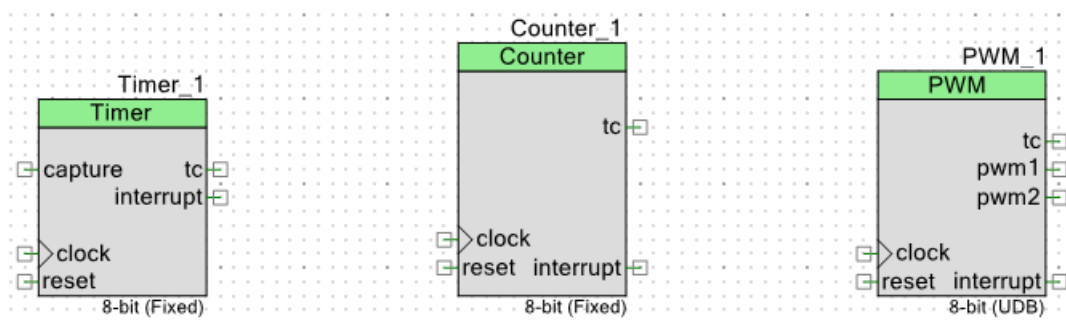
Dieser Zähler kann **intern oder extern getaktet werden**, auch ein Herunterteilen der internen Taktfrequenz ist möglich (**Prescaler**). Der Zählwert wird mit einem Referenzwert (**Compare-Register**) verglichen. Stimmen beide überein, kann ein **Ausgang** gesetzt oder ein **Interrupt ausgelöst** werden. Damit lassen sich z. B. **pulsweitenmodulierte Signale (PWM)** generieren. Viele Timer/Counter-Peripherien unterstützen mehrere Compare-Register, sodass mehrere Ereignisse definiert werden können. Für PWM gibt es oft zusätzliche Optionen, z. B. die Erzeugung **überlappungsfreier Signale** für H-Brücken.

Tritt eine Flanke an einem speziellen Eingang auf, wird der aktuelle Zählwert in ein internes **Capture-Register** gespeichert und es kann ebenfalls ein Interrupt ausgelöst werden.

Damit ergeben sich typische Anwendungsfälle:

- **Counter**, wenn externe Ereignisse/Impulse gezählt werden sollen (Frequenzzähler, Drehgeber/Quadratur-Decoder);
- **Timer**, wenn **periodische Ereignisse** bzw. **genaue Zeitbasen** (Takt-Ticks, Wecker, Input-Capture) benötigt werden;
- **PWM**, wenn über die **Pulsweite** eines digitalen Signals die mittlere Leistung bestimmt wird – z. B. bei der Steuerung von LED-Helligkeit, Motoren oder Schaltnetzteilen.

Der PSoC5LP unterscheidet daher zwischen Timer-/Counter- und PWM-Einheit, wobei je nach Fixed/UDB-Implementierung unterschiedliche Parameter einstellbar sind:



Timer (PSoC-Komponente):

- typische Anwendung: periodische Ereignisse oder Interrupts erzeugen.
- weitere Möglichkeiten: Clock-Divider, Messen von Zeitabständen (Capture).

Counter (PSoC-Komponente):

- typische Anwendung: Zählen externer Ereignisse oder Impulse.
- weitere Möglichkeiten: Clock-Divider, Frequenzmessung, Auswertung von Quadratur-Decodern (Drehgebern).

PWM (PSoC-Komponente):

- typische Anwendung: Erzeugen periodischer Signale mit einstellbarem Tastgrad (Duty Cycle).
- weitere Möglichkeiten: Leistungs- und Motorsteuerung, Schaltnetzteile, LED-Ansteuerung.

💡 **Merke:** Timer, Counter und PWM besitzen viele Gemeinsamkeiten – sie unterscheiden sich vor allem in ihren **Standardanwendungen**: Timer → Zeitbasis, Counter → Ereigniszähler, PWM → modulierte Signale.

4 Schnittstellen

Der Datenaustausch zwischen einem Mikroprozessorsystem und seiner Umwelt kann prinzipiell auf zwei verschiedene Arten erfolgen:

parallel und
seriell

Ein **paralleler Bus** besitzt **mehrere Datenleitungen** (typ. 8/16/32/64 Bit); die Übertragung erfolgt **wort-/byteweise** pro Takt. Beispiele: Speicherbusse im PC (z. B. 64 Bit pro Speicherkanal), ältere Standards wie **Parallel-ATA**.

Vorteile:

- Sehr hoher **Durchsatz** pro Takt
- Einfache Sender/Empfängerlogik (mit Bustreibern)

Nachteile:

- Viele Leitungen / Hoher Pinbedarf → **Platz/Kosten**
- Längen-/Skew-Probleme, schlecht für große Distanzen/EMV

Der **serielle Bus** besteht aus **wenigen Datenleitungen** (typ. 1 bis 6), die Datenübertragung erfolgt **bitweise**. Aus der PC-Welt können der USB-Bus, PCI-Express bzw. PCIe oder serial ATA bzw. SATA genannt werden; bei μ C: **UART, SPI, I²C, CAN, LIN**.

Vorteile:

- **Wenige Pins/Leitungen** → kompakt & kostengünstig
- **Lange Distanzen** besser realisierbar (bes. mit differentieller Übertragung)
- Datenfernübertragung per **Funk / optisch / akustisch einfacher möglich**

Nachteile:

- Protokoll-/Kodier-Overhead, ggf. höhere Latenz

Von einer **synchronen** Übertragung spricht man, wenn man neben den Datenleitungen noch eine **separate Taktleitung** hat (oder einen expliziten Taktdomänenbezug) – und die **Datenübernahme synchron zu diesem Takt** erfolgt (z.B. SPI). Bei einer **asynchronen** Übertragung fehlt die Taktleitung; Sender/Empfänger verwenden **Baudrate/Sampling**, Start-/Stop-Bits (z. B. UART).

💡 Merke: Synchrone vs. Asynchrone Übertragung

Synchron: gemeinsame Taktleitung oder Taktdomäne; Datenübernahme bei definierten Taktflanken.
Asynchron: kein gemeinsamer Takt; stattdessen Baudrate und Start-/Stop-Bits. Der Empfänger muss sich selbst auf die eingehenden Daten synchronisieren.

Schnittstellenbausteine sind das Bindeglied zwischen dem Prozessor und seiner Peripherie / seinem Speicher. Je nach Art übernehmen sie u. a.:

- **Ein-/Ausgabedaten puffern**; insbesondere bei unterschiedlichen Arbeitsgeschwindigkeiten

- Umsetzung der Daten
 - **parallel** ↔ **seriell**
 - **analog** ↔ **digital** (ADC/DAC)
 - **Wert** ↔ **Zeitinformation** (z. B. PWM)
 - **Pegelwandlung** (z. B. 5 V ↔ 3,3 V, Open-Drain)
- **Erzeugung von Takt-/Synchronisationssignale**
- **Adressierung/Arbitrierung** (z. B. Multi-Master bei I²C, Geräteadressen, Chip-Selects)
- **Fehlererkennung/-behandlung** (Parity, CRC, Timeouts, Retries)
- **Interrupt-Meldungen** an die CPU und **DMA-Unterstützung**

Unabhängig von parallel/seriell unterscheidet man die **Richtungsmodi**

- **Simplex** (Daten nur in eine Richtung)
- **Halb-Duplex** (beide Richtungen, aber abwechselnd) und
- **Voll-Duplex** (beide Richtungen gleichzeitig).

Bei der **Leitungstechnik** unterscheidet man

- **Single-Ended** (Bezug zu GND; z. B. CMOS/LVTTL, RS-232) und
- **differenziell** (Signal als Spannungsdifferenz auf einem A/B-Paar; z. B. USB, RS-485, CAN).

i Info: Differenzsignale sind **störungsempfindlicher** und für **längere Leitungen** geeignet; oft sind **Abschlusswiderstände** und kontrollierte **Leitungsimpedanz** nötig (EMV/Signal-Integrität). Zu den elektrischen Pegeln/Treibern zählen neben CMOS/LVTTL auch Open-Drain/Open-Collector (typisch I²C mit Pull-Ups) sowie spezielle Transceiver für Standards wie RS-232/RS-485. Für zuverlässige Übertragung sind - je nach Datenrate und Leitungslänge - Terminierung, Twisted-Pair/Schirmung, passende Pull-Up-Widerstände und Layout entscheidend.

4.1 UART

... ist die Abkürzung für "*Universal Asynchronous Receiver/Transmitter*") und beschreibt das **asynchrone serielle Rahmenformat** auf **Logikebene**: Daten werden mit **Startbit**, **Datenbits**, optionalem **Paritätsbit** und **Stoppbit(s)** übertragen – **ohne Taktleitung** (asynchron).

RS-232/RS-422/RS-485 sind dazu **passende elektrische Schnittstellen**, aber **nicht** die UART selbst. RS-232 ist der mit Abstand am weitesten verbreitete Standard und wurde 1960 ursprünglich für die Modem-Kommunikation entwickelt. In den letzten Jahren wurde er immer wieder überarbeitet und ergänzt (Erhöhung der max. Übertragungsgeschwindigkeit, zusätzliche Signalleitungen, ...).

Der RS-232 Standard enthält

- **elektrische Vorgaben** (Spannungspegel, Leitungskapazität, ...)

- **mechanische Vorgaben** (Steckverbindungen, Pinbelegung und -bezeichnung, ...)
- **Vorgaben zur Datenübertragung** (Geschwindigkeit, ...)

i Info: Die exakte Bezeichnung lautet EIA RS-232 (*EIA: Electronic Industries Alliance*). Bei uns weit verbreitet sind die äquivalenten Normen der CCITT (*Comité Consultatif International Téléphonique et Télégraphique*, jetzt: ITU-T) mit den Bezeichnungen V.24 (Steckverbinder und Signale) und V.28 (elektrische Pegel)... wobei man anstelle von V.24/V.28 nur V.24 liest.

RS-232 ist eine **Punkt-zu-Punkt-Verbindung** und erlaubt die Verbindung

eines **DTE** (*Data Terminal Equipments*; einem PC, μ C, ...) mit einem **DCE** (*Data Communication Equipment*; einem Modem, einem Drucker, einem Funkmodul, ...)

oder

zweier DTE mittels sog. Null-Modem-Kabel

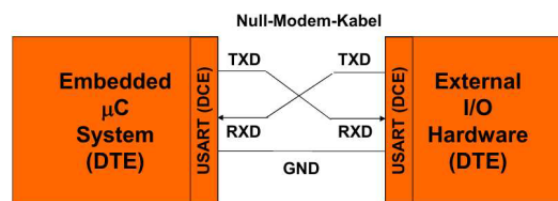
Die deutschen Begriffe / Abkürzungen von DTE sind DEE (Datenendeinrichtung) und DÜE (Datenübertragungseinrichtung).

Die Übertragung erfolgt mit Spannungswerten, die auf Masse bezogen sind (unsymmetrisch). Die Übertragung der Daten erfolgt in negativer Logik, d.h. eine Spannung im Bereich von

-15V ... -3V wird als Mark bezeichnet und symbolisiert eine log. 1
 +3V ... +15V wird als Space bezeichnet und symbolisiert eine log. 0

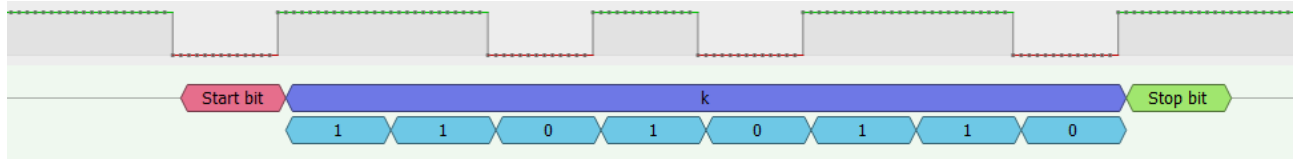
wobei die Kabellänge auf 15...20m begrenzt ist. Für RS-232 benötigt man daher **Pegelwandler** (z. B. MAX232) zwischen μ C-Pegel (3,3 V/5 V) und den RS-232-Spannungen.

Der Datenaustausch erfolgt asynchron (d.h. ohne Taktleitung) mittels TxD und RxD, alle weiteren Signalleitungen (wie z.B. CTS, RTS, ...) dienen zur Steuerung.



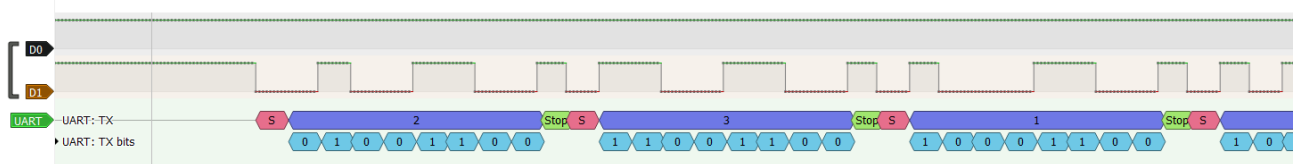
Bildquelle: Alfred Stumpf

Die Daten werden in einem Rahmen ("Frame") mit Start- und Stoppbit übertragen. Zur (zugegebenermaßen sehr einfachen) Sicherung kann ein Paritätsbit mitgesendet werden. Die **Anzahl der Datenbits** und die Grundkonfiguration der **Start-/Stopp-** und **Paritätseinstellung** sind einstellbar und müssten getrennt sowohl auf der Sende- als auch auf der Empfangsseite **ident** administriert werden. Die Angabe erfolgt oft in einer **Kurzschreibweise** der Form *DatenbitParityStoppbit* – z.B. **8N1** (8 Datenbits, No Parity, 1 Stoppbit). Ebenfalls auf beiden Seiten gleich einzustellen ist die Übertragungsrate – die sog. **Baudrate**.

Erläuterung der seriellen Datenübertragung anhand des ASCII-Zeichen "k" (Hex: 6B):

- der Idle-Zustand (Ruhezustand / nicht aktiv / frei) ist log. 1.
- die fallende (negative) Flanke symbolisiert den Beginn der Datenübertragung
- das Startbit ist stets log. 0
- danach folgen die Nutzdaten (minimal 5Bit, maximal 9 Bit); beginnend mit dem LSB
- im Anschluß kann das optionale Paritätsbit (Prüfung auf gerade oder ungerade Anzahl an log. 1) übertragen werden; ist bei der Abbildung nicht der Fall
- Stoppbit ist stets log. 1; die Übertragung kann mit einem oder zwei Stoppbits abgeschlossen werden

Die Zeitdauer eines Bits $T_{\text{bit}} = 1 / \text{Baudrate}$. Da unmittelbar nach dem Stoppbit wieder ein Startbit folgen kann, ist die minimale "idle"-Zeit zwischen 2 Datenwörtern T_{bit} bzw. $2T_{\text{bit}}$.



Bei 8N1 und einer Baudrate von 9600bps errechnet sich die maximal erreichbare Übertragungsrate an Wörtern:

$$\frac{\text{Baudrate}}{\text{Anzahl Bit pro Übertragung}} = \frac{9600 \text{ bps}}{1+8+1 \text{ Bit/Wort}} = 960 \text{ Wörter/s}$$

Mit Parität oder zwei Stopbits sinkt die Netto-Rate entsprechend.

Aufgabe: UART und GPS-NMEA-Daten

Ein GPS-Empfänger gibt seine Positionsdaten im **NMEA-Format** mit **4800 Baud, 8N1** aus. Ein typischer NMEA-Satz hat eine Länge von ca. 80 Zeichen (ASCII, je 1 Byte).

1. Wie viele Bits werden für ein Zeichen übertragen?
2. Wie viele Bits umfasst ein vollständiger NMEA-Satz?
3. Berechne die Zeit, die benötigt wird, um einen Satz bei 4800 Baud zu übertragen.
4. Angenommen, der Empfänger sendet jede Sekunde genau einen NMEA-Satz: Wie viel Prozent der Übertragungszeit pro Sekunde werden für die Daten genutzt?

Nun werden **zwei verschiedene NMEA-Sätze** (z. B. \$GPGLA und \$GPRMC) benötigt. Dafür wird die Baudrate auf **9600 Baud, 8N1** erhöht.

5. Berechne die Übertragungszeit pro Sekunde und vergleiche sie mit den Ergebnissen aus 3. und 4.

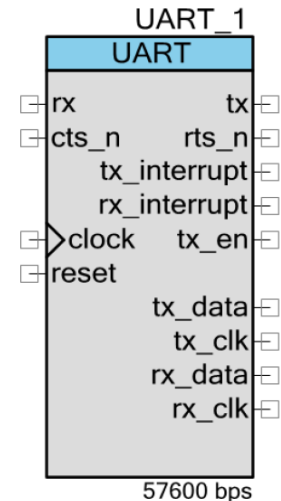
Der PSoC5LP erlaubt die Implementierung der UART-Funktionalität auf zwei Arten:

- als **Hardware-UART** mit umfangreichen Konfigurationsmöglichkeiten,
- als **Soft-UART**, bei der die Funktionalität in Software nachgebildet wird (standardmäßig nur für die Übertragung verfügbar).

Auszug aus dem Datenblatt der hardware-basierten UART:

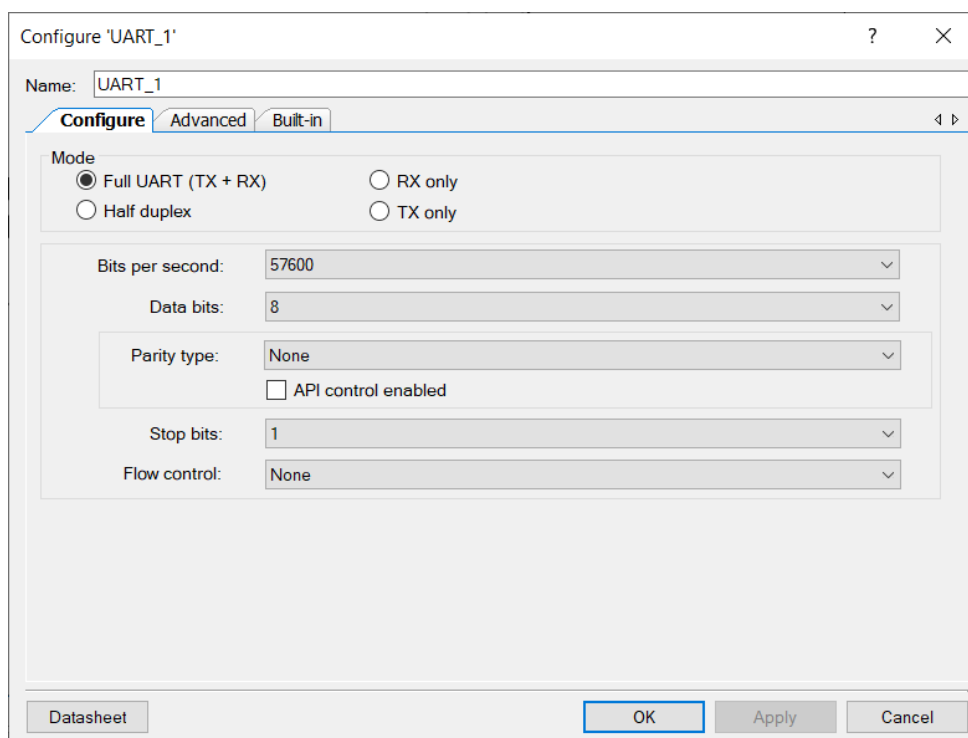
Features

- 9-bit address mode with hardware address detection
- Baud rates from 110 to 921600 bps or arbitrary up to 4 Mbps
- RX and TX buffers = 4 to 65535
- Detection of Framing, Parity, and Overrun errors
- Full Duplex, Half Duplex, TX only, and RX only optimized hardware
- Two out of three voting per bit
- Break signal generation and detection
- 8x or 16x oversampling



Full Duplex: **gleichzeitiges** Senden- und Empfangen von Daten (d.h. gleichzeitiger Datenaustausch in beiden Richtungen). *Half Duplex* ermöglicht nur das **Empfangen oder Senden** zu einem Zeitpunkt.

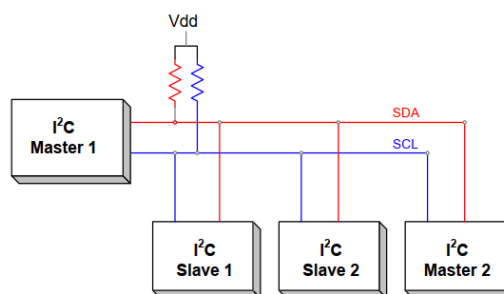
Die Einstellung der UART-Parameter kann entweder mittels GUI oder zur Laufzeit per API-Aufrufe erfolgen.



4.2 Inter-Integrated Circuit Bus - I²C

Der I²C-Bus wurde 1982 von Philips Semiconductor (heute NXP Semiconductors) entwickelt. Es handelt sich hier um einen **synchronen, seriellen und bidirektionalen** Zweidrahtbus mit **Master-Slave-Architektur**, wobei auch ein **Multimaster-Betrieb** möglich ist. I²C steht für *Inter-Integrated Circuit*; die ursprüngliche Intention bestand darin, die zunehmende Anzahl an ICs in Fernsehgeräten per Zweidrahtbus miteinander zu verbinden. Im Gegensatz zur UART, die für Punkt-zu-Punkt-Verbindungen gedacht ist, erlaubt I²C die Kommunikation von **mehreren Teilnehmern über nur zwei Leitungen**. Laut NXP existieren heute über 1000 ICs mit I²C Schnittstelle, gefertigt von mehr als 50 Firmen¹.

Ursprünglich erhob Philips Lizenzgebühren für die Implementierung von I²C. Aus diesem Grund führten Hersteller wie Atmel (heute Microchip) den **TWI-Bus (Two-Wire Interface)** ein, der technisch weitgehend identisch zu I²C ist.



Bildquelle: AN50987 von Infineon

Der I²C-Bus besteht aus der **Taktleitung SCL (Serial Clock)** und der **Datenleitung SDA (Serial Data)**. Sämtliche, am I²C Bus angeschlossene Komponenten können aktiv nur gegen Masse schalten – d.h. alle Teilnehmer besitzen **Open-Drain/-Collector-Ausgänge** – so dass zwei externe Pull-Up-Widerstände benötigt werden. Der Wert hängt von der Übertragungsgeschwindigkeit, von der Buslänge und somit von der Leitungskapazität ab und liegt meist bei 2.2kΩ ... 4.7kΩ ... 10kΩ.

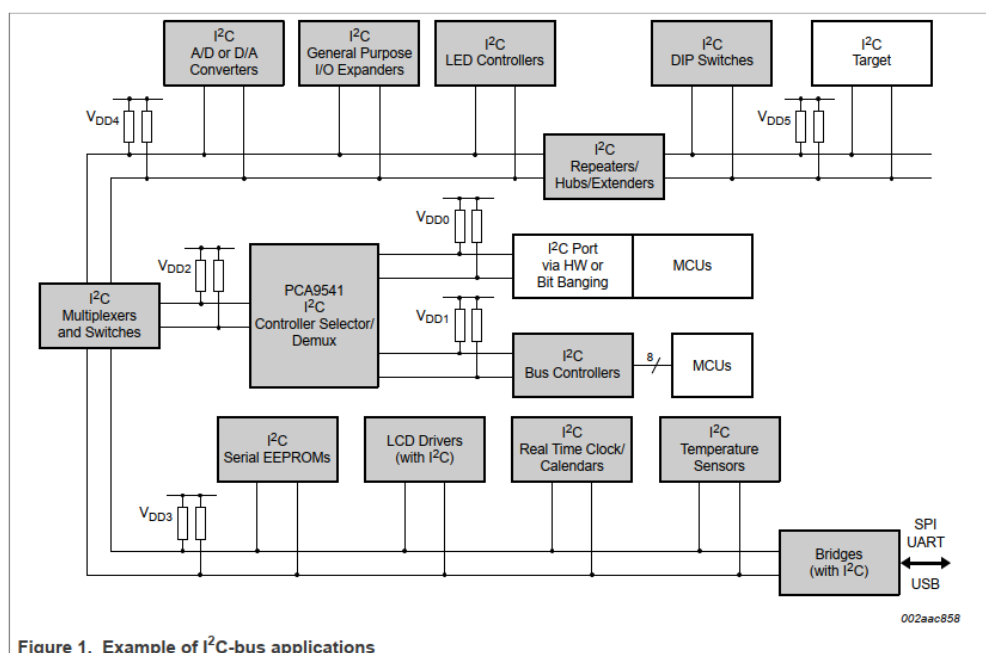


Figure 1. Example of I²C-bus applications

Bildquelle: UM10204 von NXP

1 UM10204, I²C-bus specification and user manual, Rev. 7 – Oktober 2021; <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>

Der Bustakt wird **immer vom Master vorgegeben** (wobei an einem I²C Bus auch mehrere Master angeschlossen sein können). Wird nicht über dem Bus kommuniziert, dann liegen SDA und SCL auf High. D.h. nur während einer Kommunikation liegt auch ein Taktsignal auf SCL.

Modus	Taktfrequenz
Standard Mode	100kHz
Fast Mode	400kHz
Fast Mode Plus	1MHz
High Speed Mode	3,4MHz
Ultra Fast Mode (*)	5 MHz

(*) Anmerkung – unterstützt nur unidirektionalen Datenaustausch

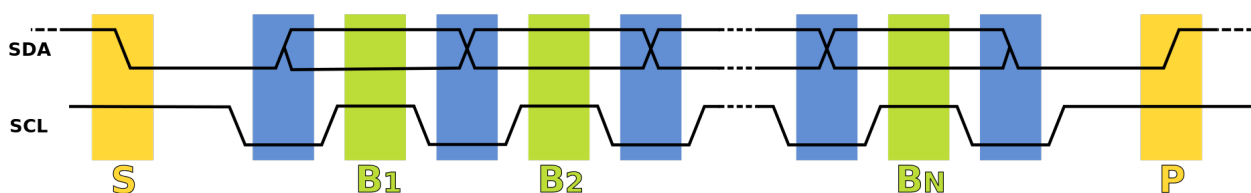
Die Frequenzangabe ist die **jeweils maximal erlaubte Taktfrequenz**; d.h. diese kann auch niedriger sein. Es gibt jedoch I²C Komponenten, die eine minimale Taktfrequenz für eine korrekte Funktion benötigen (z.B. ADC oder DAC). Entsprechende Infos findet man immer im Datenblatt. Der Standard sieht auch *mixed speed modes* vor.

Benötigt ein Slave mehr Zeit, als er durch die Taktfrequenz vom Master vorgegeben bekommt, kann er die Taktleitung auf LOW ziehen und damit die Kommunikation mit dem Master vorübergehend anhalten – Fachbegriff: *Clock Stretching*.

Je nach VDD arbeitet der I²C-Bus mit 3.3V oder 5V (typisch). Für Standard/Fast wurde LOW mit $< 0,3 \cdot VDD$ und HIGH mit $> 0,7 \cdot VDD$ festgelegt; bei Fast-mode Plus (1 MHz) ist $V_{IH} \geq 0,5 \cdot VDD$ zulässig; Auskunft gibt immer das Datenblatt.

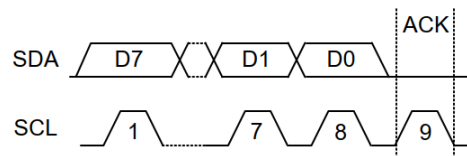
Zusätzlich können I²C Slaves einen Interrupt-Ausgang besitzen, der mit dem Interrupt-System des Masters verbunden wird.

Die Datenübertragung erfolgt per SDA Leitung in serieller Form – wobei die Übertragung mit einer **Start-** oder einer **Repeated-Start-Indikation** beginnt und mit einer **Stopp-Indikation** abgeschlossen wird. Zwischen S(tart) und P (Stopp) werden die einzelnen Bits übertragen.



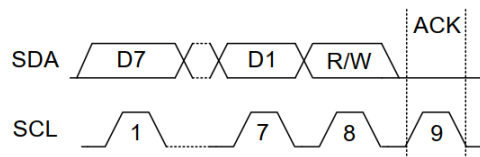
Bildquelle: de.wikipedia.org

Wichtig: Bei Start oder Stopp **ändert sich SDA, während SCL auf HIGH ist** (die Repeated-Start-Bedingung entspricht einer Start-Indikation). Bei der Übertragung der einzelnen Bits darf sich der **SDA-Zustand nur während der LOW-Phase von SCL ändern**; d.h. während SCL HIGH ist, müssen die einzelnen Datenbits stabil anliegen. Nach der Übertragung von 8 Datenbits sendet im 9. Taktzyklus die Gegenseite eine **Rückmeldung (Acknowledgement)**. Wird hier SDA auf LOW gezogen, so entspricht dies einem ACK (*acknowledge*); bei HIGH einem NAK (*not acknowledge*).



Bildquelle: AN50987 von Infineon

Jede Slave-Komponente wird über eine 7-bit breite **Adresse** ausgewählt (in einer späteren Protokoll-Überarbeitung wurde auch eine abwärtskompatible 10-bit-Adressierung eingeführt). Das 8. Bit gibt an, ob eine **Lese- oder Schreiboperation** (Read/Write) erfolgt.



Bildquelle: 7-bit-Adressierung; AN50987 von Cypress / Infineon

i Info: 7-Bit- vs. 8-Bit-Adressangaben

Die Adressangabe in Datenblättern von I²C-Komponenten ist nicht einheitlich. Einige Hersteller geben die Adresse als **7-Bit-Wert** an, andere als **8-Bit-Wert**, bei dem das *Read/Write-Bit* bereits enthalten ist (z. B. 0x84/0x85).

Beispiel: Hat ein Baustein die 7-Bit-Adresse **0x20**, dann entspricht dies 0x40 für eine Schreiboperation und 0x41 für eine Leseoperation (8-Bit-Notation). In Zweifelsfällen helfen die **Timing-Diagramme** im Datenblatt, die Klarheit über die tatsächliche Adresse geben.



Aufgabe: I²C-Adresse (Praxisbeispiel BME280 - Feuchtigkeits-/Temperatur- und Drucksensor)

Im Datenblatt des Sensors **BME280** sind die I²C-Adressen mit **0xEC/0xED** angegeben.

1. Notiere die **8-Bit-Muster** für Schreib- und Leseoperation.
2. Leite daraus die **7-Bit-Adresse** ab.
3. Informiere dich: Warum gibt es bei vielen Breakout-Boards die Möglichkeit, die Adresse des BME280 per Pin auf **0x77** zu ändern?

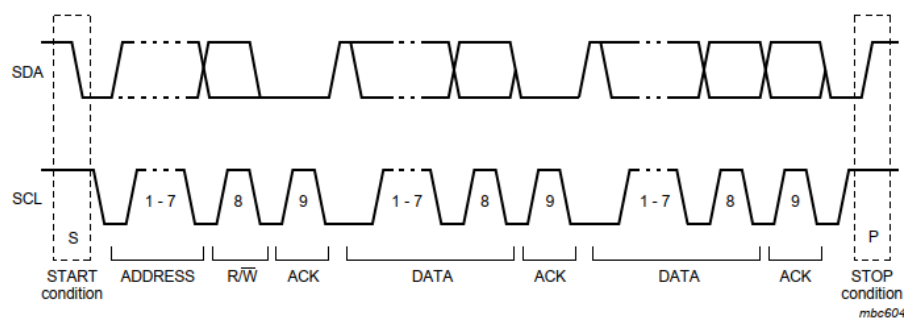
Die 7-bit-Adresse des Slaves ist prinzipiell festgelegt und besteht aus einer 4-bit Gruppenkennung, gefolgt von 3 Bit, die die Gerätefunktion repräsentieren. Philips hat damals auch berücksichtigt, dass an einem I²C Bus einige Gerätefunktionen mehrfach benötigt/angeschlossen werden können (z.B. Speicher) – daher sind in diesem Fall die unteren 3bit per Hardware-Beschaltung des jeweiligen Slaves einstellbar.

Auszug aus der I²C-Bus Allocation Table von Philips:

GROUP ⁽¹⁾			TYPE NUMBER	DESCRIPTION
Group 0 (0000)				
0	0	0	-	General call address
X	X	X	-	Reserved addresses
Group 1 (0001)				
1	A1	A0	SAA2530	ADR/DMX digital receiver
1	A1	A0	TDA8045	QAM-64 demodulator
Group 2 (0010)				
0	0	A0	SAA4700/T	VPS dataline processor
0	0	A0	SAA5233	Dual standard PDC decoder
0	0	1	SAA5243	Computer controlled teletext circuit
0	0	1	SAA5244	Integrated VIP and teletext
0	0	1	SAA5245	525-line teletext decoder/controller
0	0	1	SAA5246A	Integrated VIP and teletext
0	0	1	SAA5249	VIP and teletext controller
0	A1	A0	CCR921	RDS/RBDS decoder

Group A (1010)				
A2	A1	A0	PCF8570	256 × 8-bit static RAM
A2	A1	A0	PCF8522/4	512 × 8-bit CMOS EEPROM
A2	A1	A0	PCA8581/C	128 × 8-bit EEPROM
A2	A1	A0	PCF8582/A	256 × 8-bit EEPROM
A2	A1	P0	PCX8594	512 × 8-bit CMOS EEPROM
A2	P1	P0	PCX8598	1024 × 8-bit CMOS EEPROM

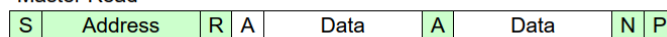
Ein Datentransfer per I²C sieht prinzipiell wie folgt aus:



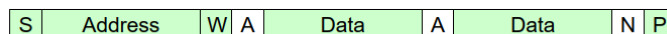
Bildquelle: UM10204 von NXP

bzw. in einer anderen Darstellungsform:

Master Read

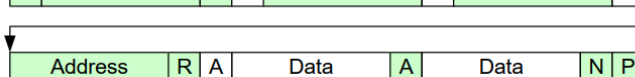
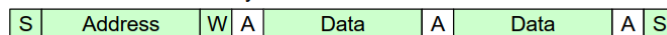


Master Write



Repeated Start Example:

Master Write Followed by Master Read



Bildquelle: AN50987 von Cypress / Infineon

Master Read – d.h. der Master liest von einer Slave-Komponente: Der Master beginnt mit der Start-Bedingung, gefolgt von der 7bit-Adresse des Slave und einem $R/\overline{W}$ =HIGH (lesend). Nach diesen 8 Bit sendet der adressierte Slave ein ACK – als Zeichen, dass die adressierte Komponente sich auch "angesprochen" fühlt. In den nächsten 8 Taktzyklen sendet der Slave seine Daten an den Master, beginnend mit dem MSB; im 9. Taktzyklus sendet der Master ein ACK. In den nächsten 8 Taktzyklen sendet der Slave die weiteren Daten – dieses Mal sendet der Master ein NACK als Zeichen, dass kein weiterer Lesezyklus mehr erfolgt. Die I²C Kommunikation wird mit einer Stopp-Bedingung beendet.

Master Write – d.h. der Master schreibt auf eine Slave-Komponente. Der Ablauf ist dem Lesen sehr ähnlich, nur mit dem Unterschied dass $R/\overline{W}$ =LOW (schreibend) ist. Nachdem der Slave das erste ACK sendet, überträgt der Master nun Daten an den Slave – im 9. Taktzyklus antwortet der Slave dann mit einem ACK. Nach der zweiten Datenübertragung sendet der Slave ein NACK und der Master beendet die I²C Kommunikation mit einer Stopp-Bedingung.

Als Beispiel für eine "**Repeated-Start-Bedingung**" wird hier der Ablauf "Master Write followed by Master Read" gezeigt.

i Info: Exkurs: SMBus

Eine sehr starke Verwandtschaft zum I²C – Bus besitzt der **System Management Bus** (abgekürzt SM-Bus, SMBus oder SMB), der 1995 von Intel eingeführt und auf vielen Motherboards vorzufinden ist. Wird die Taktleitung für mehr als 35ms auf LOW gehalten, so bedeutet dies ein Reset für alle angeschlossenen Slaves. Ebenfalls wurde eine minimale Taktfrequenz von 10kHz festgelegt. Es gibt noch weitere Unterschiede – auf diese soll hier jedoch nicht eingegangen werden. Nicht alle I²C-Geräte sind SMBus-kompatibel (und umgekehrt), vieles ist aber interoperabel.

Im Folgenden werden Datenblatt-Auszüge verschiedener I²C-Komponenten vorgestellt. Da viele dieser Bausteine von mehreren Herstellern gefertigt werden, wurden bewusst Datenblätter unterschiedlicher Anbieter ausgewählt. Die Beispiele decken typische Anwendungsbereiche des I²C-Busses ab: **I/O-Erweiterung** (PCF8574), **Sensorik** (AS6200, BME280), **Speicher** (24C512), **Zeitbasis** (DS1307), **Anzeige** (PCF2119x), **Treiber** (PCA9685) und **Bus-Management** (PCA9646).

- PCF8574 8-Bit I/O Expander - Auszug Datenblatt Texas Instruments

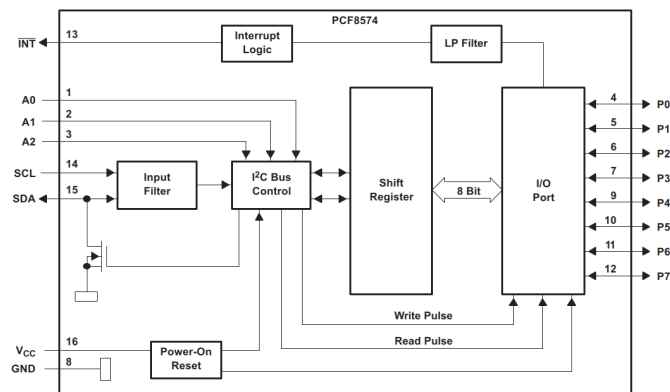
3 Description

This 8-bit input/output (I/O) expander for the two-line bidirectional bus (I²C) is designed for 2.5-V to 6-V V_{CC} operation.

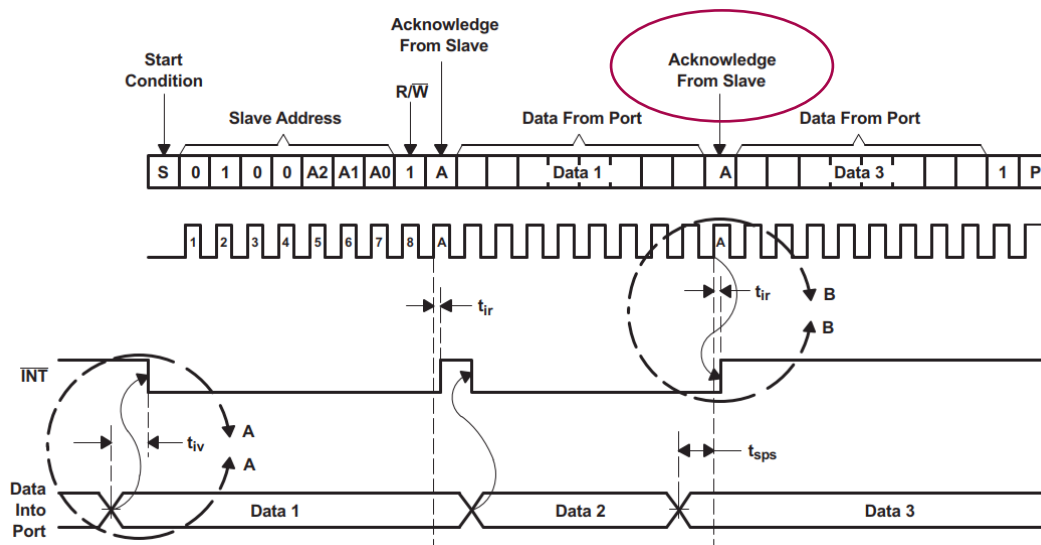
The PCF8574 device provides general-purpose remote I/O expansion for most microcontroller families by way of the I²C interface [serial clock (SCL), serial data (SDA)].

The device features an 8-bit quasi-bidirectional I/O port (P0–P7), including latched outputs with high-current drive capability for directly driving LEDs. Each quasi-bidirectional I/O can be used as an input or output without the use of a data-direction control signal. At power on, the I/Os are high. In this mode, only a current source to V_{CC} is active.

8.2.1 Simplified Block Diagram of Device



BYTE	BIT							
	7 (MSB)	6	5	4	3	2	1	0 (LSB)
I ² C slave address	L	H	L	L	A2	A1	A0	R/W
I/O data bus	P7	P6	P5	P4	P3	P2	P1	P0



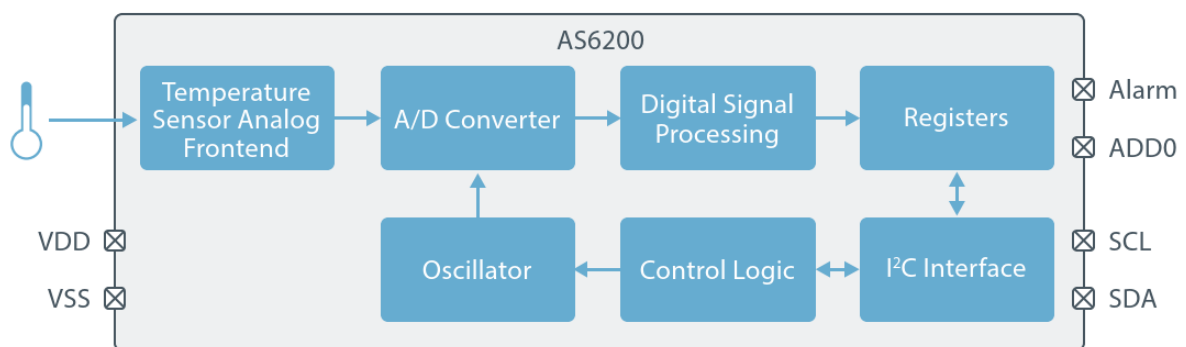
- **AS6200 Temperatursensor** – Auszug Datenblatt ams

The AS6200 IC is a high accuracy temperature sensor system that communicates via a 2 wire digital bus with other devices. It consists of a Si bandgap temperature sensor, an ADC and a digital signal processor.

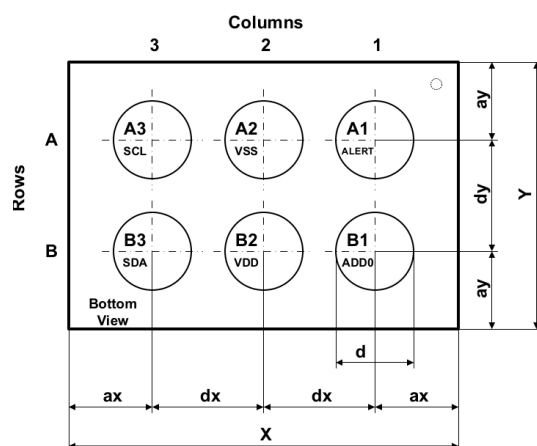
It has a very high temperature accuracy ($\pm 0.4^{\circ}\text{C}$ for AS6200) and an ultra-low power consumption (low operation and quiescent current) which makes it ideally suited for mobile/battery powered applications.

The AS6200 is an easy to integrate and use solution, featuring an factory calibrated sensor, integrated linearization and the possibility to use 2 different I²C addresses, enabling to use two AS6200 devices on one bus.

Additionally the AS6200 temperature sensor system also features an alert functionality, which triggers e.g. an interrupt to protect devices from excessive temperatures.



Address	Symbol	Register	Description
0x0	TVAL	Temperature Register	Contains the temperature value
0x1	CONFIG	Configuration Register	Configuration settings of the temperature sensor
0x2	TLOW	T _{LOW} Register	Low temperature threshold value
0x3	THIGH	T _{HIGH} Register	High temperature threshold value



Symbol ⁽¹⁾	Dimension [μm]		
	Min	Typ	Max
X	1450		1530
Y	960		1040
ax		325	
dx		400	
ay		280	
dy		400	
d		250	
Thickness (w.o. balls)		378	

• 24C512 – 512Kbit EEPROM – Auszug Datenblatt ON Semiconductor

Description

The CAT24C512 is a EERPOM Serial 512-Kb I²C internally organized as 65,536 words of 8 bits each.

It features a 128-byte page write buffer and supports the Standard (100 kHz), Fast (400 kHz) and Fast-Plus (1 MHz) I²C protocol.

Write operations can be inhibited by taking the WP pin High (this protects the entire memory).

External address pins make it possible to address up to eight CAT24C512 devices on the same bus.

On-Chip ECC (Error Correction Code) makes the device suitable for high reliability applications.

Features

- Supports Standard, Fast and Fast-Plus I²C Protocol
- 1.8 V to 5.5 V Supply Voltage Range
- 128-Byte Page Write Buffer
- Hardware Write Protection for Entire Memory
- Schmitt Triggers and Noise Suppression Filters on I²C Bus Inputs (SCL and SDA)
- Low Power CMOS Technology
- 1,000,000 Program/Erase Cycles
- 100 Year Data Retention
- Industrial and Extended Temperature Range
- 8-pin, SOIC, TSSOP, 8-pad UDFN and 8-ball WLCSP Packages
- These Devices are Pb-Free, Halogen Free/BFR Free and are RoHS Compliant

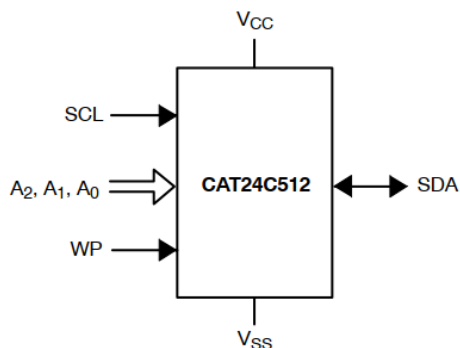


Figure 1. Functional Symbol

ON Semiconductor®

www.onsemi.com

TSSOP-8
Y SUFFIX
CASE 948AL

UDFN-8
HU5 SUFFIX
CASE 517BU

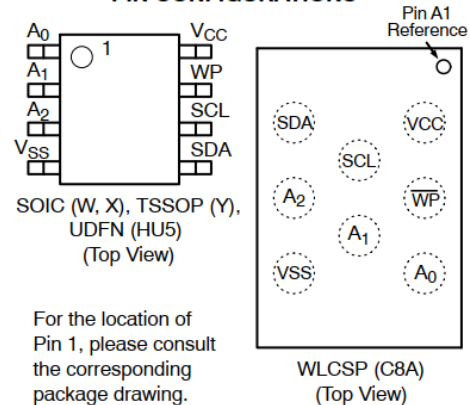
SOIC-8
X SUFFIX
CASE 751BE

SOIC-8
W SUFFIX
CASE 751BD

SOIC-8 WIDE
X SUFFIX
CASE 751BE

WLCSP-8
C8A SUFFIX
CASE 567JL

PIN CONFIGURATIONS



PIN FUNCTION

Pin Name	Function
A ₀ , A ₁ , A ₂	Device Address
SDA	Serial Data
SCL	Serial Clock
WP	Write Protect
V _{CC}	Power Supply
V _{SS}	Ground

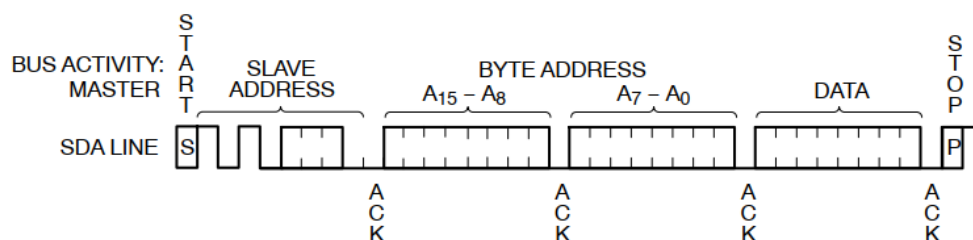


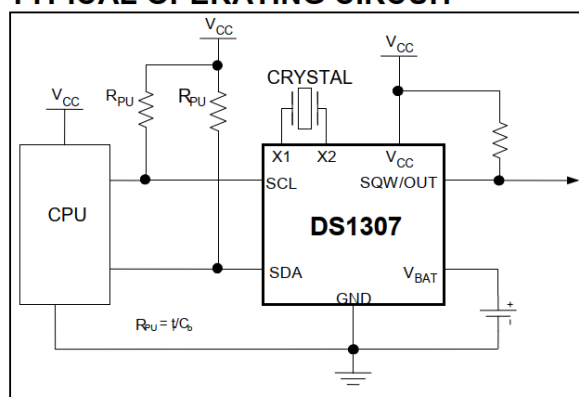
Figure 6. Byte Write Timing

• DS1307 RTC (Real-Time-Clock) – Auszug Datenblatt Maxim Integrated

GENERAL DESCRIPTION

The DS1307 serial real-time clock (RTC) is a low-power, full binary-coded decimal (BCD) clock/calendar plus 56 bytes of NV SRAM. Address and data are transferred serially through an I²C, bidirectional bus. The clock/calendar provides seconds, minutes, hours, day, date, month, and year information. The end of the month date is automatically adjusted for months with fewer than 31 days, including corrections for leap year. The clock operates in either the 24-hour or 12-hour format with AM/PM indicator. The DS1307 has a built-in power-sense circuit that detects power failures and automatically switches to the backup supply. Timekeeping operation continues while the part operates from the backup supply.

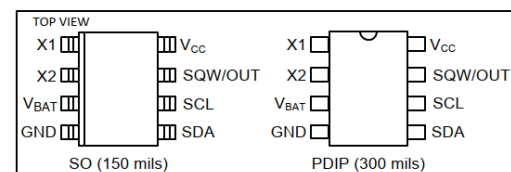
TYPICAL OPERATING CIRCUIT



BENEFITS AND FEATURES

- Completely Manages All Timekeeping Functions
 - Real-Time Clock Counts Seconds, Minutes, Hours, Date of the Month, Month, Day of the Week, and Year with Leap-Year Compensation Valid Up to 2100
 - 56-Byte, Battery-Backed, General-Purpose RAM with Unlimited Writes
 - Programmable Square-Wave Output Signal
- Simple Serial Port Interfaces to Most Microcontrollers
 - I²C Serial Interface
- Low Power Operation Extends Battery Backup Run Time
 - Consumes Less than 500nA in Battery-Backup Mode with Oscillator Running
 - Automatic Power-Fail Detect and Switch Circuitry
- 8-Pin DIP and 8-Pin SO Minimizes Required Space
- Optional Industrial Temperature Range: -40°C to +85°C Supports Operation in a Wide Range of Applications
- Underwriters Laboratories® (UL) Recognized

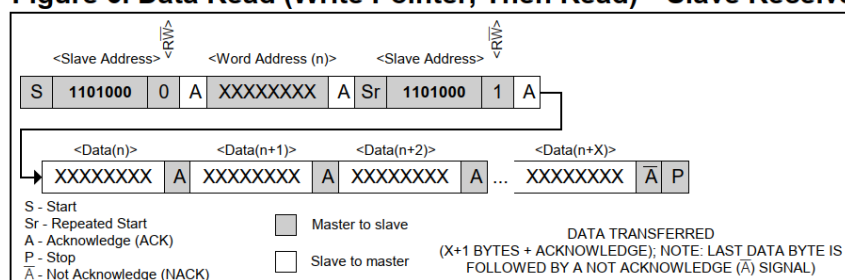
PIN CONFIGURATIONS



ADDRESS	BIT 7	BIT 6		BIT 5	BIT 4	BIT 3		BIT 2	BIT 1	BIT 0	FUNCTION	RANGE
00h	CH	10 Seconds				Seconds				Seconds	00–59	
01h	0	10 Minutes				Minutes				Minutes	00–59	
02h	0	12	10 Hour	10 Hour	Hours				Hours	1–12 +AM/PM 00–23		
		24	PM/ AM									
03h	0	0	0	0	0	DAY			Day	01–07		
04h	0	0	10 Date			Date				Date	01–31	
05h	0	0	0	10 Month	Month				Month	01–12		
06h	10 Year					Year				Year	00–99	
07h	OUT	0	0	SQWE	0	0	RS1	RS0	Control	—		
08h–3Fh										RAM 56 x 8	00h–FFh	

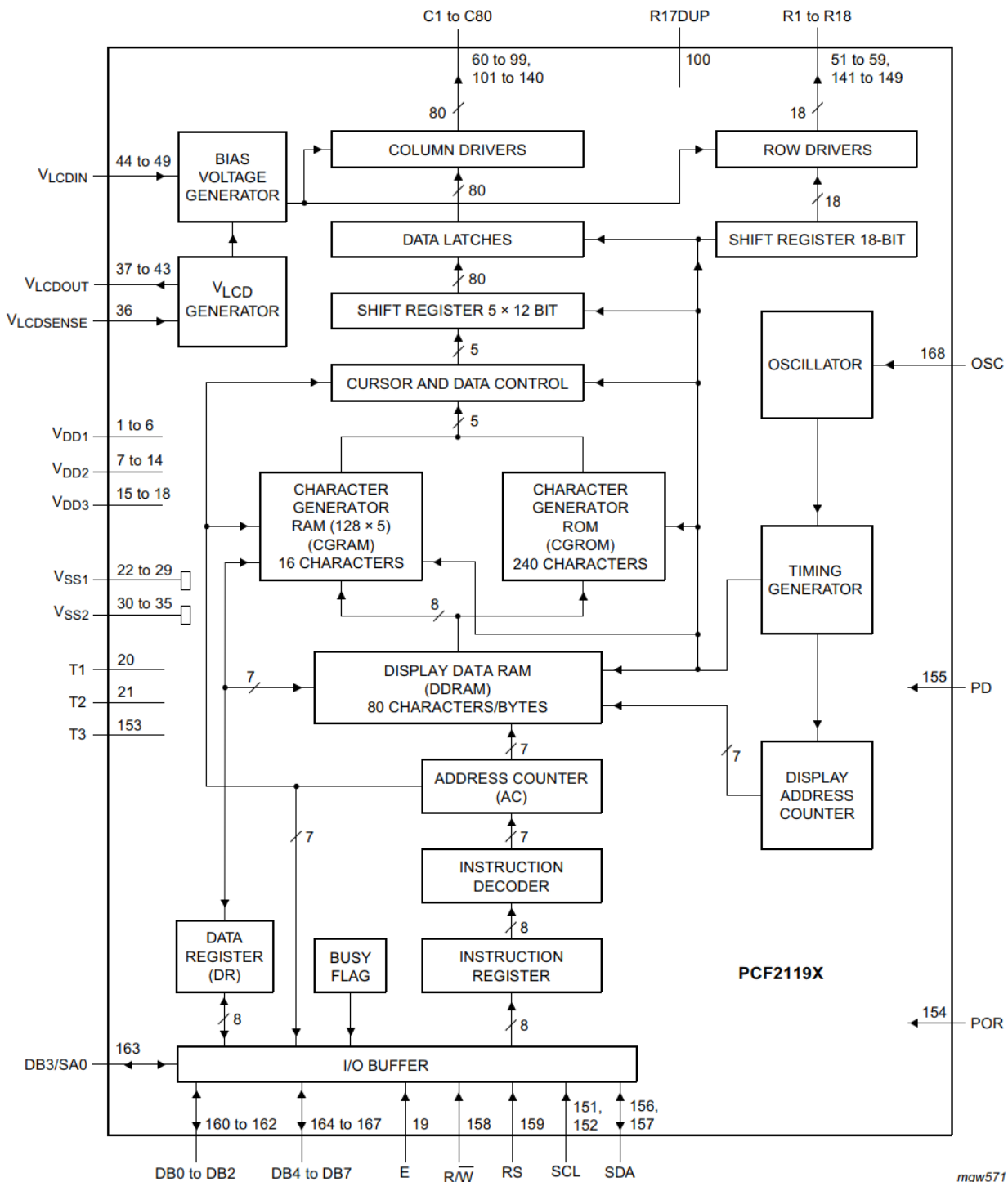
0 = Always reads back as 0.

Figure 6. Data Read (Write Pointer, Then Read)—Slave Receive and Transmit



- PCF2119x LCD Controllers/drivers – Auszug Datenblatt NXP

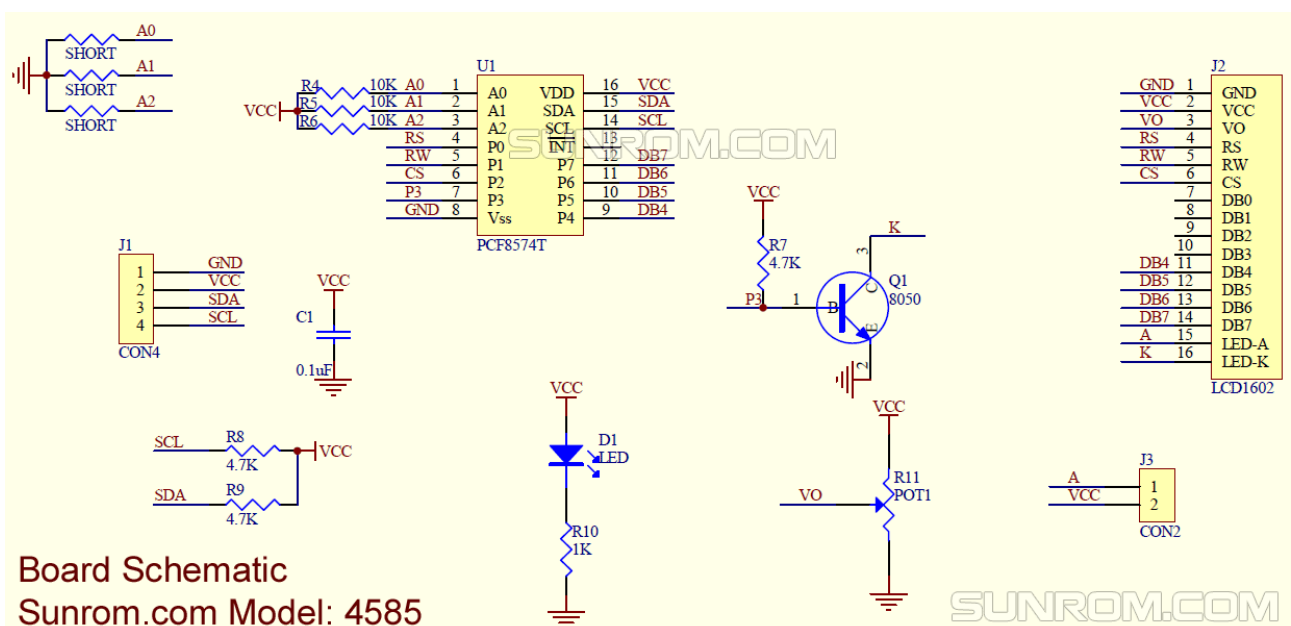
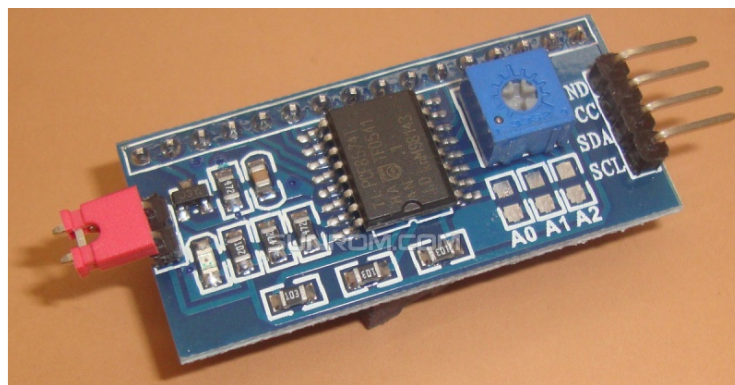
The PCF2119x is a low power CMOS¹ LCD controller and driver, designed to drive a dot matrix LCD display of 2-lines by 16 characters or 1-line by 32 characters with 5 × 8 dot format. All necessary functions for the display are provided in a single chip, including on-chip generation of LCD bias voltages, resulting in a minimum of external components and lower system current consumption. The PCF2119x interfaces to most microcontrollers via a 4-bit or 8-bit bus or via the 2-wire I²C-bus. The chip contains a character generator and displays alphanumeric and kana (Japanese) characters.



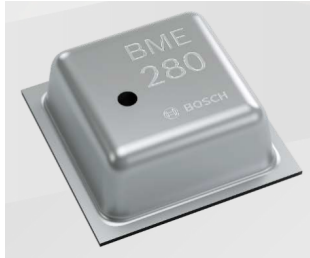
ACHTUNG – hierbei handelt es sich um KEIN natives I²C-LCD-Display!



Bildquelle: <http://arduino-basics.com/lcd-display/>

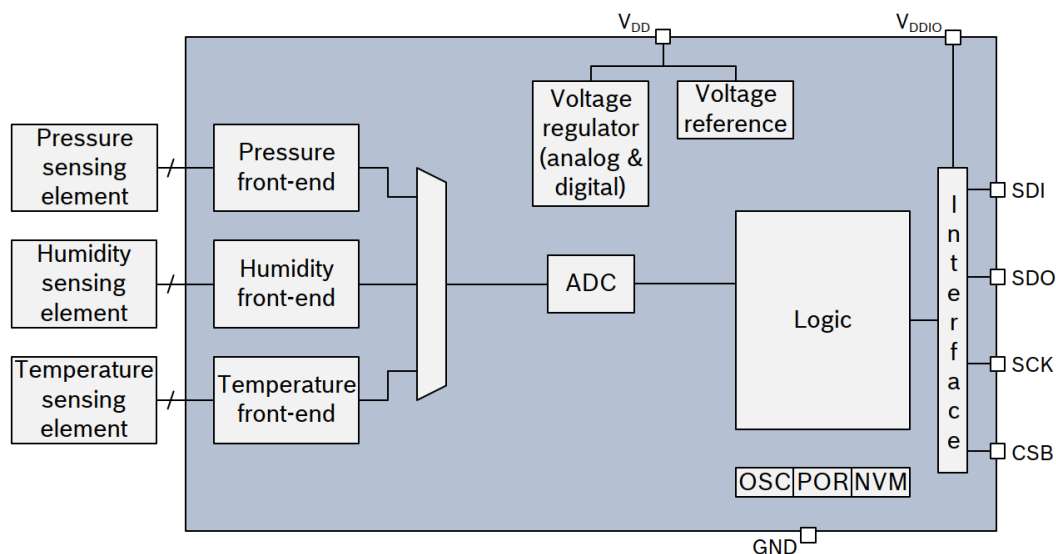


- **BME280 Feuchtigkeits-/Temperatur- und Drucksensor** – Auszug Datenblatt Bosch



Key features

- Package 2.5 mm x 2.5 mm x 0.93 mm metal lid LGA
- Digital interface I²C (up to 3.4 MHz) and SPI (3 and 4 wire, up to 10 MHz)
- Supply voltage V_{DD} main supply voltage range: 1.71 V to 3.6 V
V_{DDIO} interface voltage range: 1.2 V to 3.6 V
- Current consumption 1.8 μ A @ 1 Hz humidity and temperature
2.8 μ A @ 1 Hz pressure and temperature
3.6 μ A @ 1 Hz humidity, pressure and temperature
0.1 μ A in sleep mode
- Operating range -40...+85 °C, 0...100 % rel. humidity, 300...1100 hPa
- Humidity sensor and pressure sensor can be independently enabled / disabled
- Register and performance compatible to Bosch Sensortec BMP280 digital pressure sensor
- RoHS compliant, halogen-free, MSL1



The I²C interface uses the following pins:

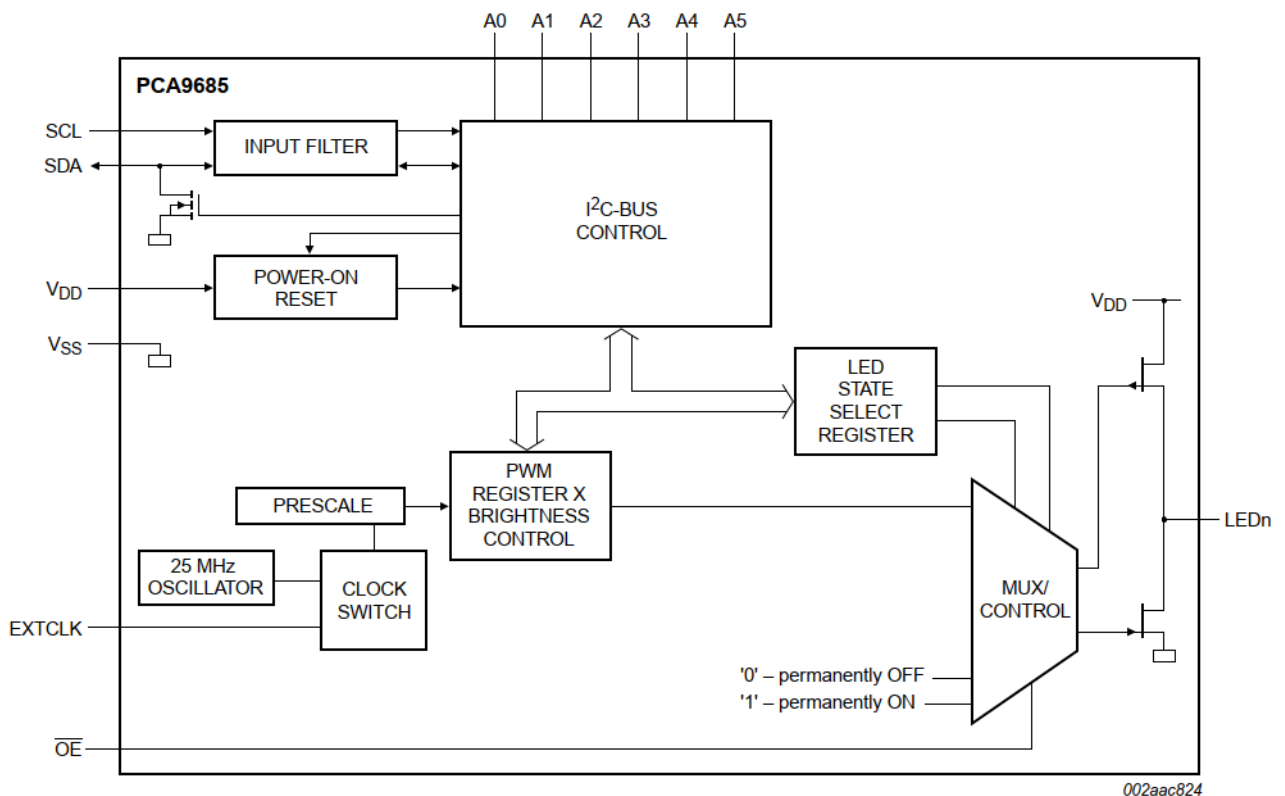
- SCK: serial clock (SCL)
- SDI: data (SDA)
- SDO: Slave address LSB (GND = '0', V_{DDIO} = '1')

CSB must be connected to V_{DDIO} to select I²C interface. SDI is bi-directional with open drain to GND: it must be externally connected to V_{DDIO} via a pull up resistor. Refer to chapter 7 for connection instructions.

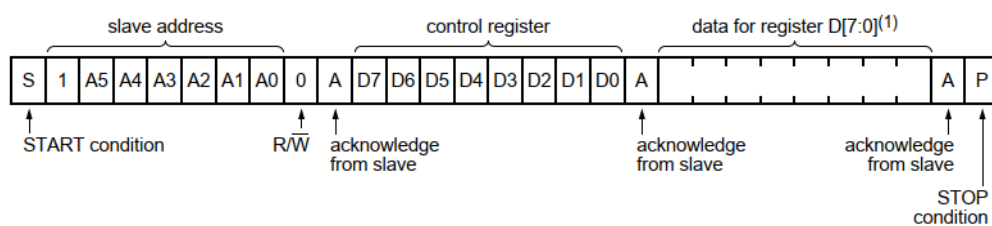
- **PCA9685 16-channel, 12-bit PWM Fm+ I2C-bus LED controller** – Auszug Datenblatt NXP

The PCA9685 is an I²C-bus controlled 16-channel LED controller optimized for Red/Green/Blue/Amber (RGBA) color backlighting applications. Each LED output has its own 12-bit resolution (4096 steps) fixed frequency individual PWM controller that operates at a programmable frequency from a typical of 24 Hz to 1526 Hz with a duty cycle that is adjustable from 0 % to 100 % to allow the LED to be set to a specific brightness value. All outputs are set to the same PWM frequency.

Each LED output can be off or on (no PWM control), or set at its individual PWM controller value. The LED output driver is programmed to be either open-drain with a 25 mA current sink capability at 5 V or totem pole with a 25 mA sink, 10 mA source capability at 5 V. The PCA9685 operates with a supply voltage range of 2.3 V to 5.5 V and the inputs and outputs are 5.5 V tolerant. LEDs can be directly connected to the LED output (up to 25 mA, 5.5 V) or controlled with external drivers and a minimum amount of discrete components for larger current or higher voltage LEDs.



Remark: Only one LED output shown for clarity.

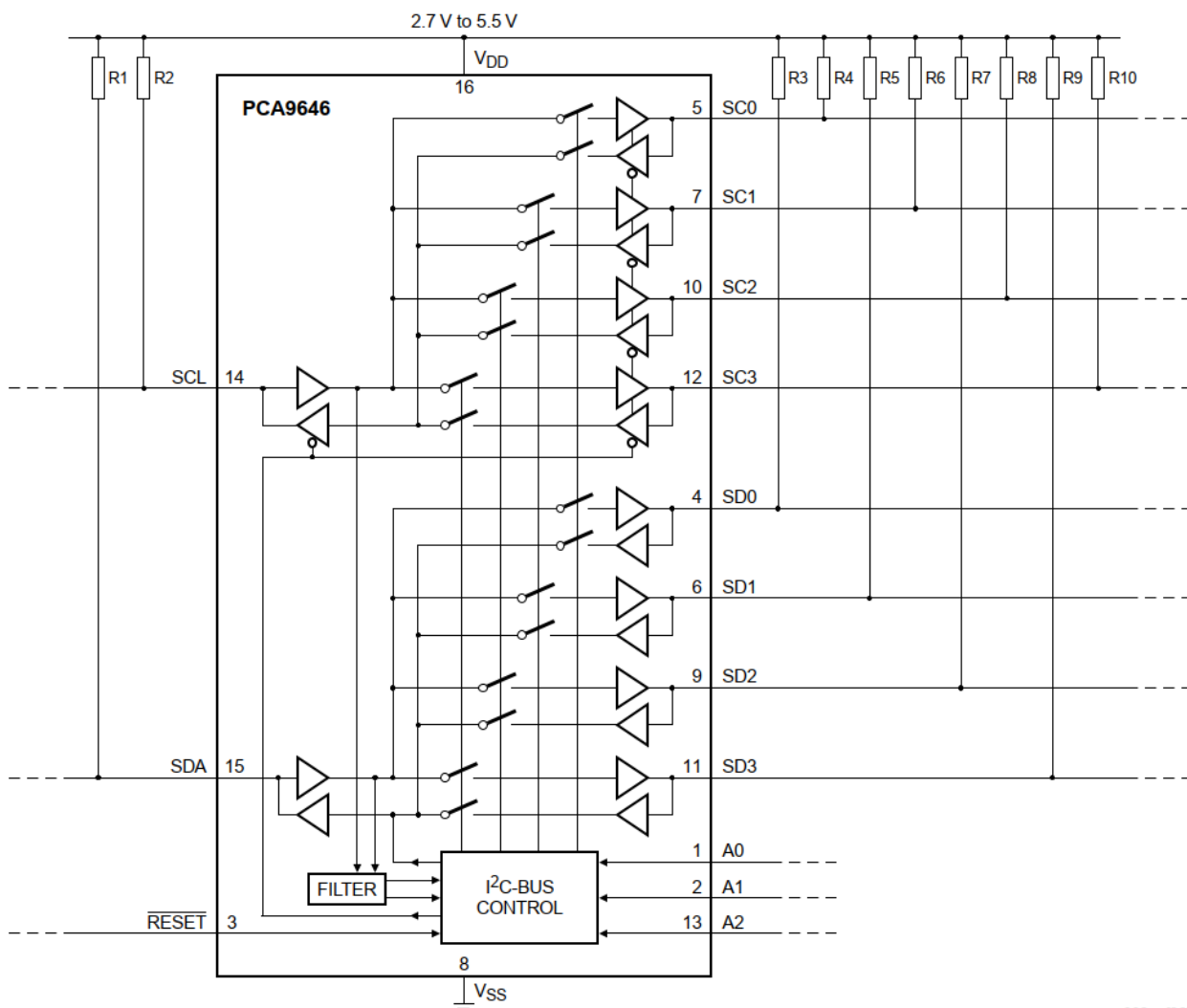


- **PCA9646 4-channel 2-wire bus switch** – Auszug Datenblatt NXP

The PCA9646 is a monolithic CMOS integrated circuit for 2-wire bus buffering and switching in applications including I²C-bus, SMBus, PMBus, and other systems based on similar principles.

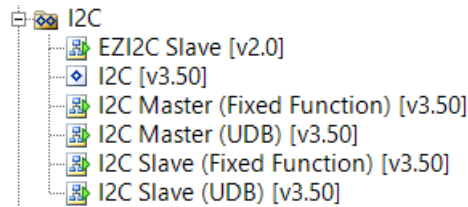
Each of the four outputs may be independently enabled in any combination as determined by the contents of the programmable control register. Each I/O is impedance isolated from all others, thus allowing a total of five branches of 2-wire bus with the maximum specified load (e.g., 5×400 pF for Fm+ I²C-bus at 1 MHz, or 5×4 nF at lower frequencies) ([Ref. 1](#)). More than one PCA9646 may be used in series, providing a substantial fan-out capability.

The PCA9646 includes a unidirectional buffer for the clock signal, and a bidirectional buffer for the data signal. The direction of the clock signal may also be set by the contents of the programmable control register. Clock stretching and timing must always be under control of the master device.



002aaf367

Der PSoC5LP bietet eine ausgesprochen flexible Unterstützung des I²C-Busses. Neben der Nutzung als **Master** oder **Slave** ist auch der **Multi-Master-Betrieb** vorgesehen. Für typische Slave-Anwendungen steht darüber hinaus die **EZI2C-Komponente** zur Verfügung, die einen I²C-Speicherbaustein (I²C ↔ SRAM) nachbildet, selbstständig arbeitet und sich besonders einfach einsetzen lässt.



Die Implementierung kann wahlweise über **dedizierte Hardwareblöcke** oder über das **UDB-Subsystem** (*Universal Digital Blocks*) erfolgen, wodurch sich – im Gegensatz zu vielen klassischen Mikrocontrollern mit meist nur ein bis zwei I²C-Einheiten – mehrere unabhängige I²C-Bus-Systeme realisieren lassen.

Die I²C-Komponenten sind **vielfältig parametrierbar** und können sowohl über die grafische Konfigurationsumgebung in PSoC Creator als auch **zur Laufzeit per API** gesteuert werden.

Ein weiterer Vorteil im Laborbetrieb ist, dass der **KitProg-Programmieradapter** auch als **USB ↔ I²C-Brücke** fungiert – damit lassen sich externe I²C-Bausteine bequem über den PC ansteuern.

KI-Aufgabe: Ausblick I³C

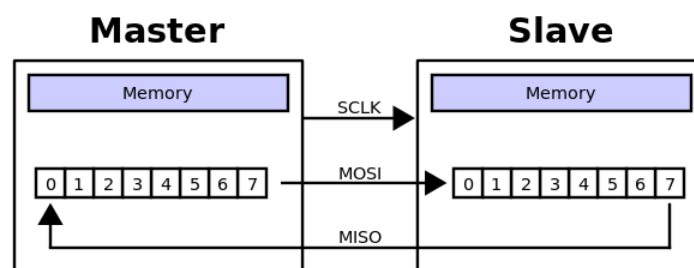
Bitte eine KI um eine kurze technische Erklärung, was der **I³C-Bus** ist und wie er sich vom klassischen I²C-Bus unterscheidet. Achte dabei besonders auf folgende Punkte:

- Übertragungsrate
- Energiebedarf
- Kompatibilität zu I²C

Diskutiere anschließend mit deinem Banknachbarn, in welchen Anwendungen der Einsatz von I³C sinnvoll wäre, und halte die Ergebnisse in **eigenen Worten** schriftlich fest.

4.3 Serial Peripheral Interface Bus - SPI

... wurde 1987 von Motorola (jetzt: NXP Semiconductor) entwickelt und ist ein **synchroner, serieller, vollduplexfähiger Bus** nach dem Master-Slave-Prinzip. Motorola hat SPI nie zu einem Patent angemeldet, dieses Bussystem wurde auch nie in eine Norm gegossen. Daher existieren auch unterschiedliche Bezeichnungen für Anschlüsse und unterschiedliche Übertragungsmodi – aber aufgrund der Tatsache, dass es sich hier um ein sehr einfaches Bussystem handelt (Grundprinzip: Schieberegister), ist es extrem weit verbreitet und wird von vielen Herstellern unterstützt.



Bildquelle: https://en.wikipedia.org/wiki/Serial_Peripheral_Interface

Prinzipiell besteht der SPI Bus aus

- einer Datenleitung für die Kommunikation Master → Slave
- einer Datenleitung für die Kommunikation Slave → Master
- einer Taktleitung

Als Bezeichnungen findet man

- **MOSI** (Master Out / Slave In)
- **MISO** (Master In / Slave Out)
- **SCK** oder **SCLK**

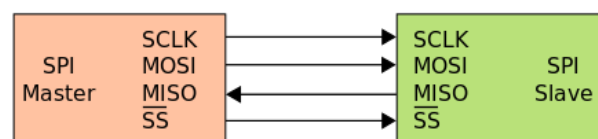
i Info: Terminologien

Da die Begriffe *Master* und *Slave* als historisch belastet gelten, gibt es Bestrebungen, alternative Bezeichnungen einzuführen. Häufig findet man daher die neutralen Bezeichnungen **SDO/SO** (Serial Data Out) und **SDI/SI** (Serial Data In). Die Richtungsangabe erfolgt jeweils aus Sicht des Busteilnehmers (SDO ist mit SDI zu verbinden). Vereinzelt tauchen auch die Bezeichnungen **POCI** (Peripheral Out / Controller In) und **PICO** (Peripheral In / Controller Out) auf.

An einem SPI-Bus kann (theoretisch) eine unlimitierte Anzahl an Slaves angeschlossen werden; die Auswahl des Slave-Bausteins erfolgt über eine dedizierte Chip-Select-Leitung oder Slave-Select-Leitung und diese ist in der Regel Low-Aktiv; d.h. $\overline{CS}$ bzw. $\overline{SS}$. Mehrere Slaves können alternativ auch per Kaskadierung (*Daisy-Chain*) an einen Master angebunden werden.

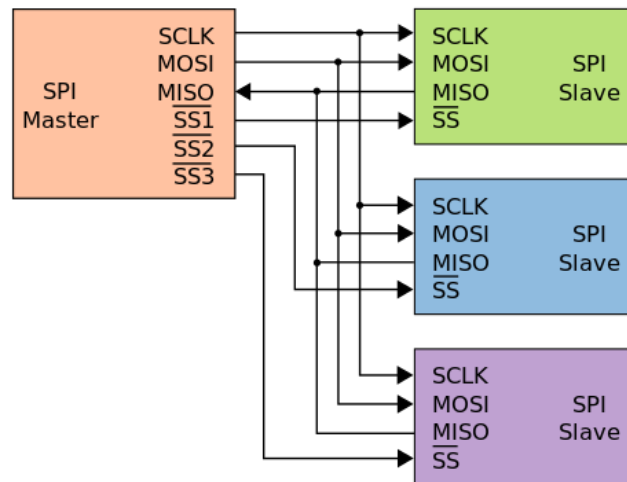
SPI-Bus mit einem Master und...

... einem Slave:



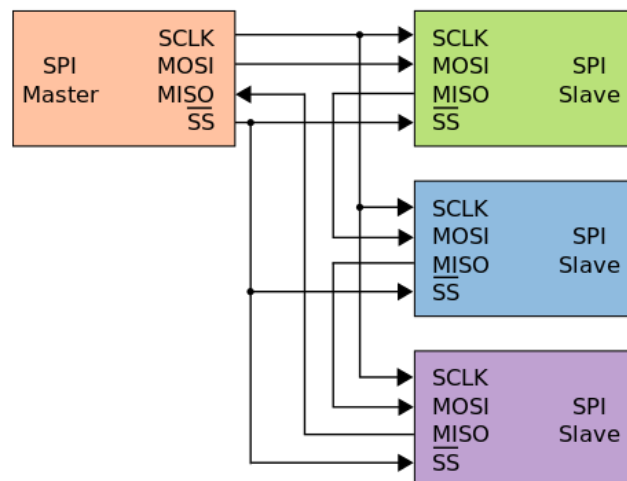
Bildquelle: https://de.wikipedia.org/wiki/Serial_Peripheral_Interface

... mehreren Slaves, wobei hier jeder Slave per Select-Leitung ausgewählt wird:



Bildquelle: https://de.wikipedia.org/wiki/Serial_Peripheral_Interface

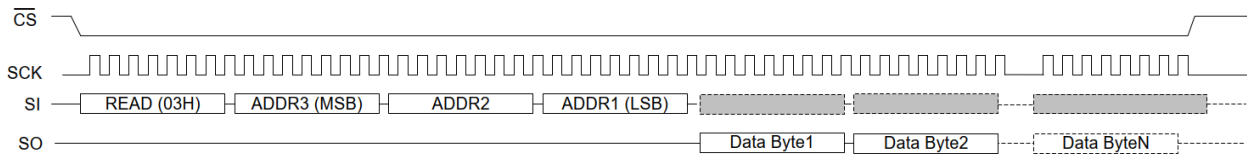
... mehreren Slaves in Kaskadierung:



Bildquelle: https://de.wikipedia.org/wiki/Serial_Peripheral_Interface

i Info: MOSI und MISO können auch zu einer einzelnen, bidirektionalen SDAT Leitung zusammengefasst werden; in diesem Fall spricht man vom **SPI Bidirectional Mode**. Der Master muss hier die Richtung der gemeinsamen Datenleitung (SDAT) umschalten. Man verliert dadurch zwar die Full-Duplex – Fähigkeit, spart jedoch eine Datenleitung.

Die folgende Abbildung zeigt exemplarisch einen Lesezugriff auf einen SPI-F-RAM-Baustein:



Bildquelle: AN89659 Interfacing SPI F-RAM von Infineon (Cypress)

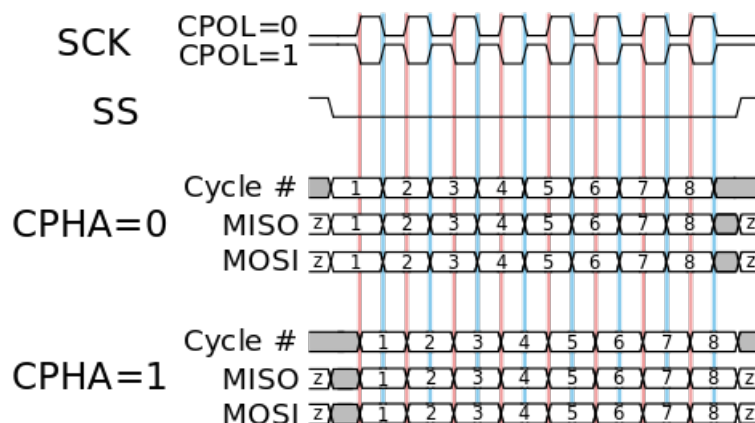
Der Master zieht die $\overline{\text{CS}}$ -Leitung auf Low und legt das Taktsignal an SCK an. Das Kommunikationsprotokoll des F-RAM-Bausteins schreibt vor, dass das erstes Byte ein sog. Opcode sein muss. Damit weiß das F-RAM, ob z.B. ein Lese- oder Schreibzugriff erfolgen soll oder ob z.B. das Statusregister angesprochen werden soll. In diesem Zeitverlauf ist der Opcode = READ (03h), sodass die nächsten 3 Bytes die zu adressierende Speicheradresse enthalten. Sobald für den F-RAM-Baustein feststeht, ab welcher Speicheradresse gelesen werden soll, werden die Daten auf SO ausgegeben.

Der SPI-Bus bietet viele Freiheitsgrade, und daher ist es essentiell, das Datenblatt des Slave-Bausteins sorgfältig zu studieren um den Master entsprechend korrekt parametrisieren zu können; z.B.

- **Ruhepegel Taktleitung** HIGH oder LOW → Polarität des Taktsignals
- **Übernahme der Daten bei steigender oder fallender Flanke**
- **Anzahl der Bits**, die übertragen/ausgetauscht werden (meist: 8)
- **Beginn** mit dem **LSB** oder **MSB** (meist: MSB)
- **Taktfrequenz**

Die Kombination aus dem Ruhepegel des Taktes (Clock Polarity – CPOL) und der Flankeninformation (Clock Phase – CPHA) ergeben **vier Modi**:

CPOL	CPHA	Mode
0	0	0
0	1	1
1	0	2
1	1	3



Bildquelle: https://de.wikipedia.org/wiki/Serial_Peripheral_Interface

Viele Slave-Bausteine verlangen: $\overline{CS}$ bleibt LOW während dem Datentransfer und die Daten werden oft erst bei steigender Flanke von $\overline{CS}$ übernommen/verarbeitet. Daher wird die $\overline{CS}$ -Leitung auch benötigt, wenn nur ein Slave-Baustein an dem SPI-Bus angeschlossen ist.

i Info: SPI ohne Hardware-Unterstützung

Besitzt ein Mikrocontroller keine interne SPI-Einheit, so kann die Funktionalität auch in Software nachgebildet werden (*Bitbanging*) oder durch einen externen SPI-Controller bereitgestellt werden, der über den Systembus angebunden ist.

Auf dem Markt gibt es eine Vielzahl an SPI-Slave-Bausteinen; Sensoren, Speicher, AD/DA-Umsetzer, digitale Potentiometer, Anzeigenbausteine / LCD-Displays, Gegenüber dem I²C Bus hat der SPI-Bus einen **höheren Durchsatz (Full-Duplex, Push-Pull-Ausgänge, keine Begrenzung auf 8Bit, keine Slave-Adressierung)**, allerdings auch **Nachteile** – so fehlt z.B. ein Slave-Acknowledgement, wie dies beim I²C Bus der Fall ist. Im Worst-Case-Fall kann der Master seine Daten an einen nicht-existenten Slave senden ohne dies zu merken. **Robustheit** erreicht man ggf. durch **Readback, CRC/Checksumme auf Protokollebene** oder **Geräte-Statusbits** (z. B. Busy/Ready).

Varianten des SPI

- **Microwire (μ Wire):** kann als Vorgängerversion betitelt werden, ist ein Trademark von National Semiconductor, unterstützt ausschließlich Half-Duplex und SPI-Mode 0.
- **Microwire/Plus:** ist eine Erweiterung von μ Wire, mit Full-Duplex und SPI-Mode 0 und 1.
- **Dual SPI:** wird i.d.R. von SPI Speicherbausteinen (meist ROM) unterstützt, wenn große Datenmengen überwiegend in eine Richtung übertragen werden. Hierbei ist kein Full-Duplex notwendig; d.h. per Konfigurationskommando kann MOSI/MISO in SIO0/SIO1 (Serial IO) umgeschaltet werden. Es können dann 2 Bits / Taktzyklus gleichzeitig übertragen werden.
- **Quad SPI:** Erweiterung von Dual SPI und besitzt zwei zusätzliche Datenleitungen, so dass 4 Bits / Taktzyklus gleichzeitig übertragen werden können (SIO0 ... SIO3). Wie bei Dual SPI muss per Protokollkonfiguration in den QSPI-Modus geschaltet werden, da sich sonst eine QSPI-Komponente wie eine "normale" SPI-Komponente verhält.

Stellvertretend werden auf den nächsten Seiten einige unterschiedliche SPI-Slave-Bausteine mit ihren jeweiligen Datenblatt-Auszügen vorgestellt. Die Auswahl deckt typische Anwendungsbereiche ab: **Anzeige** (MAX7219), **Analogtechnik** (MCP41xy), **Kommunikation** (ENC28J60) und **Speicher** (SD/MMC-Karten).

• **MCP413x/415x/423x/425x Digital Potentiometer** – Auszug Datenblatt Microchip

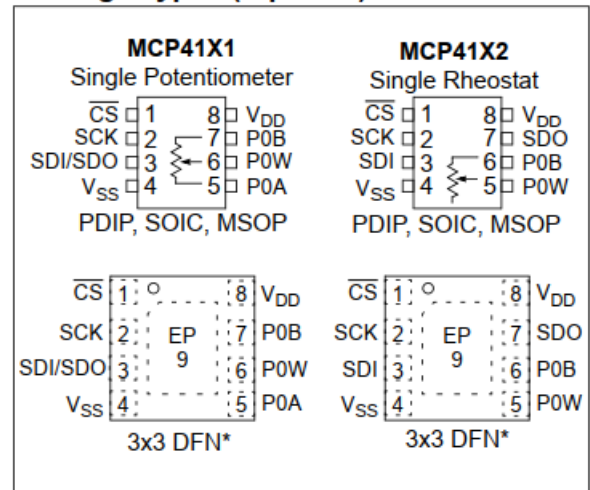
Features

- Single or Dual Resistor Network options
- Potentiometer or Rheostat configuration options
- Resistor Network Resolution
 - 7-bit: 128 Resistors (129 Steps)
 - 8-bit: 256 Resistors (257 Steps)
- R_{AB} Resistances options of:
 - 5 k Ω
 - 10 k Ω
 - 50 k Ω
 - 100 k Ω
- Zero Scale to Full-Scale Wiper operation
- Low Wiper Resistance: 75 Ω (typical)
- Low Tempco:
 - Absolute (Rheostat): 50 ppm typical (0°C to 70°C)
 - Ratiometric (Potentiometer): 15 ppm typical
- SPI Serial Interface (10 MHz, modes 0,0 & 1,1)
 - High-Speed Read/Writes to wiper registers
 - SDI/SDO multiplexing (MCP41X1 only)
- Resistor Network Terminal Disconnect Feature

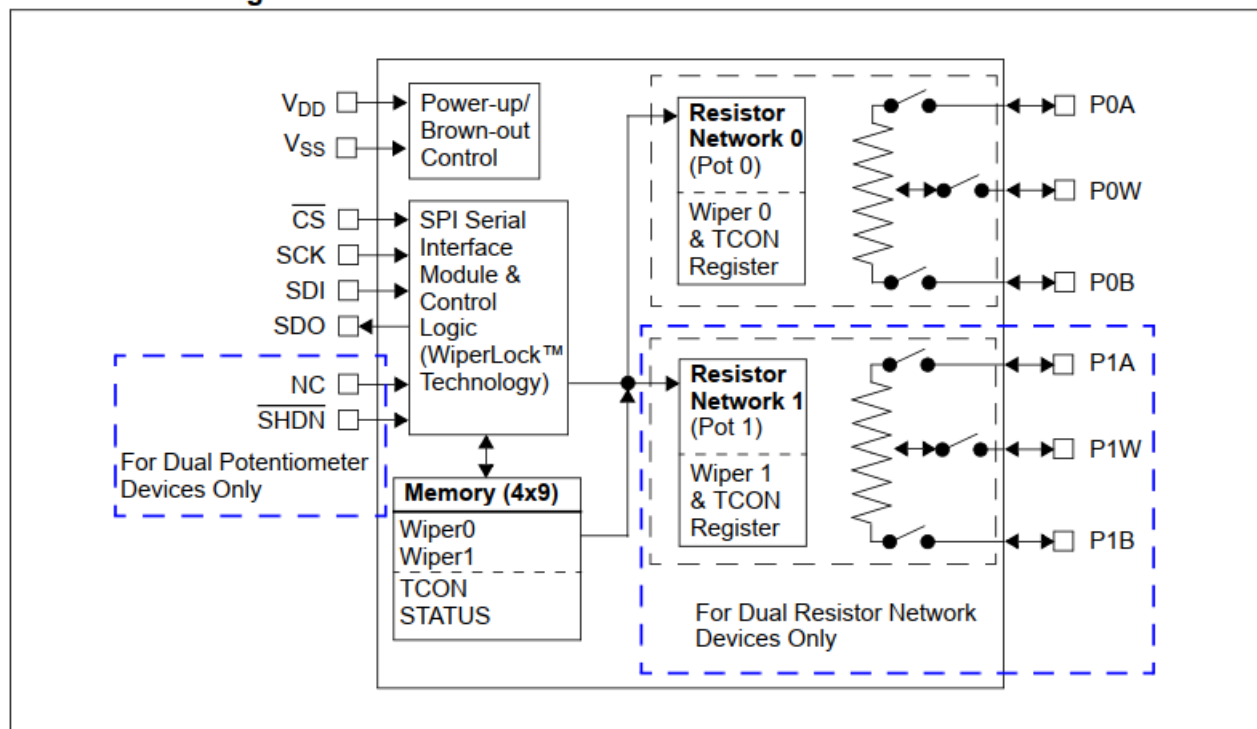
Description

The MCP41XX and MCP42XX devices offer a wide range of product offerings using an SPI interface. This family of devices support 7-bit and 8-bit resistor networks, and Potentiometer and Rheostat pinouts.

Package Types (top view)



Device Block Diagram



ENC28J60 Standalone Ethernet Controller – Auszug Datenblatt Microchip

Ethernet Controller Features

- IEEE 802.3™ Compatible Ethernet Controller
- Fully Compatible with 10/100/1000Base-T Networks
- Integrated MAC and 10Base-T PHY
- Supports One 10Base-T Port with Automatic Polarity Detection and Correction
- Supports Full and Half-Duplex modes
- Programmable Automatic Retransmit on Collision
- Programmable Padding and CRC Generation
- Programmable Automatic Rejection of Erroneous Packets
- SPI Interface with Clock Speeds Up to 20 MHz

28-Pin SPDIP, SSOP, SOIC

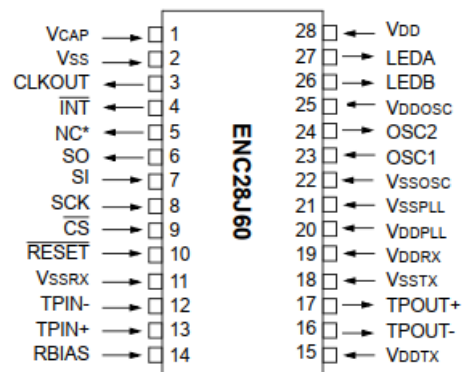
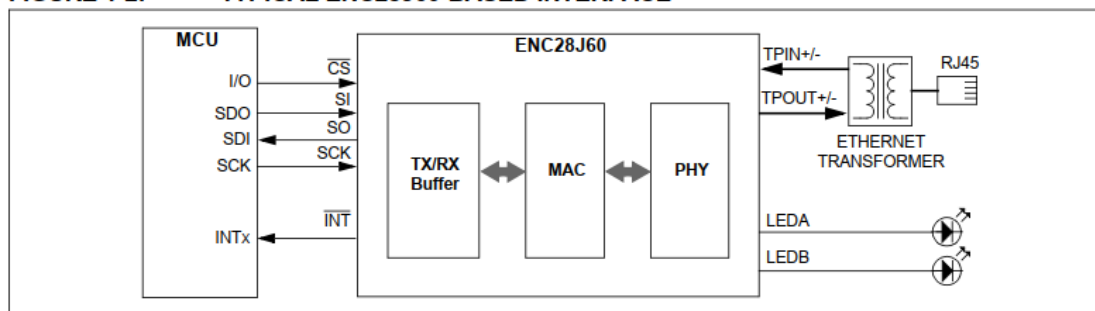


FIGURE 1-2: TYPICAL ENC28J60-BASED INTERFACE

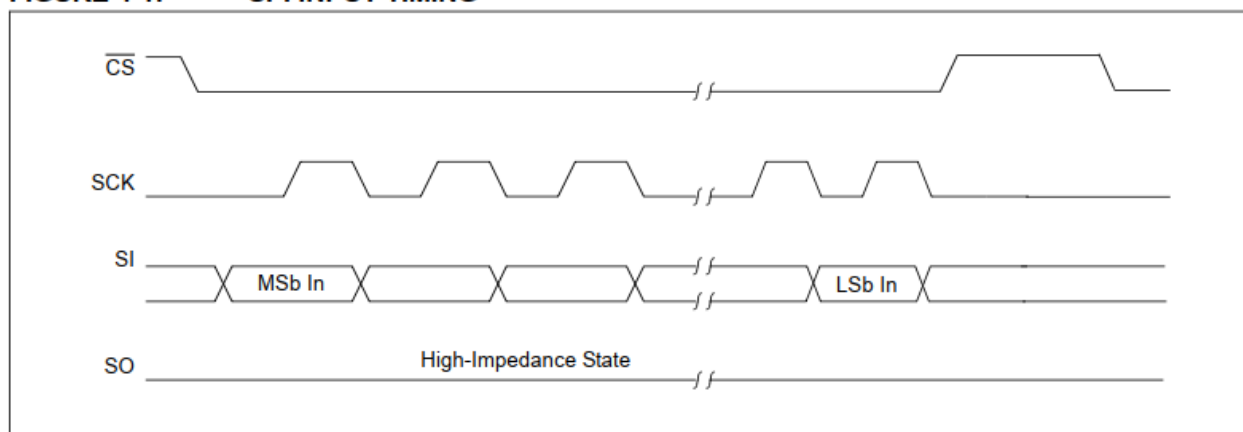


4.1 Overview

The ENC28J60 is designed to interface directly with the Serial Peripheral Interface (SPI) port available on many microcontrollers. The implementation used on this device supports SPI mode 0,0 only. In addition, the SPI port requires that SCK be at Idle in a low state; selectable clock polarity is not supported.

held low while any operation is performed and returned high when finished.

FIGURE 4-1: SPI INPUT TIMING



Im PSoC5LP lassen sich SPI-Schnittstellen flexibel implementieren – wahlweise als Master- oder als Slave-Komponente.

Kurzer Auszug aus dem Datenblatt der SPI-Master-Komponente:

[...] The SPI Master component provides an industry-standard, 4-wire master SPI interface. It can also provide a 3-wire (bidirectional) SPI interface. Both interfaces support all four SPI operating modes, allowing communication with any SPI slave device. In addition to the standard 8-bit word length, the SPI Master supports a configurable 3- to 16-bit word length for communicating with nonstandard SPI word lengths. [...]

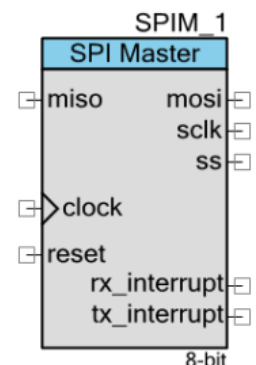


Figure 3. Firmware Controlled Slave Selects

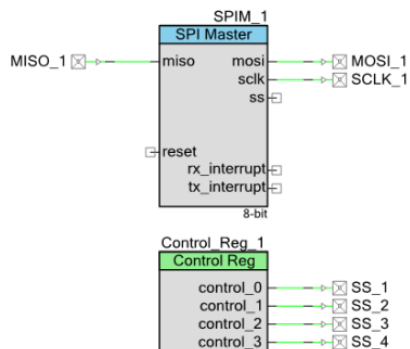
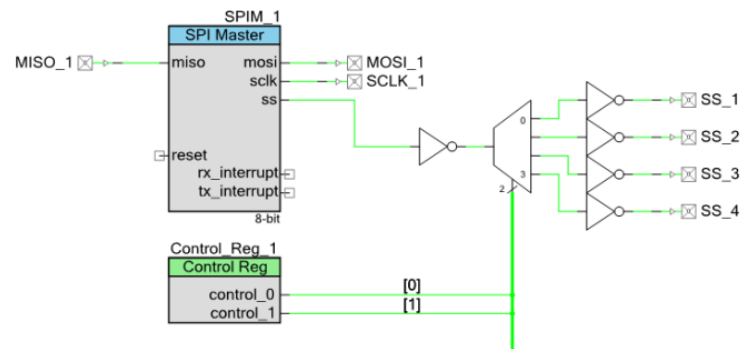
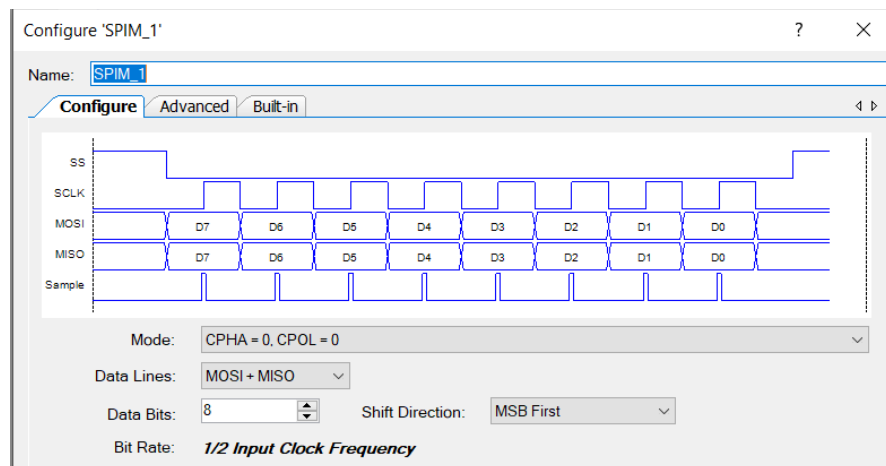


Figure 4. Slave Select Output to Demultiplexer



Die grafische Oberfläche in PSoC Creator erlaubt eine komfortable Konfiguration der SPI-Parameter (z. B. Takt, Mode, Wortlänge):



KI-Aufgabe: SD-Karten im SPI-Modus

Bitte eine KI, dir zu erklären, wie **SD-Karten im SPI-Modus an einen Mikrocontroller angebunden werden können**. Welche Leitungen werden benötigt? Wie erfolgt die Initialisierung der Karte? Welche Rolle spielt die Blockgröße bei der Datenübertragung?

Recherchiere zusätzlich mit Hilfe der KI nach **C-Bibliotheken und Beispielprojekten**, die den Zugriff auf SD-Karten über SPI implementieren. Notiere die Ergebnisse in **eigenen Worten** und diskutiere mit deinem Banknachbarn, welche Vorteile der SPI-Modus im Vergleich zum nativen SD-Bus-Modus für μ C-Anwendungen bietet.

5 Direct Memory Access - DMA

Stellen wir uns vor, ein Mikrofon nimmt Sprache oder Musik auf. Bereits bei einer Abtastrate von **44,1 kHz** (CD-Qualität) entstehen pro Sekunde **44 100 Abtastwerte**, die kontinuierlich im Speicher abgelegt werden müssen. Bei einer Wortbreite von **16 Bit** ergibt das rund **88 kByte pro Sekunde** bei Mono-Aufnahme – oder etwa **176 kByte pro Sekunde** bei Stereo. Würde die CPU jedes einzelne Sample übertragen, wäre sie fast ausschließlich mit diesem Datentransfer beschäftigt – für die eigentliche Signalverarbeitung (Filterung, Effekte, Kompression) bliebe kaum Rechenzeit übrig.

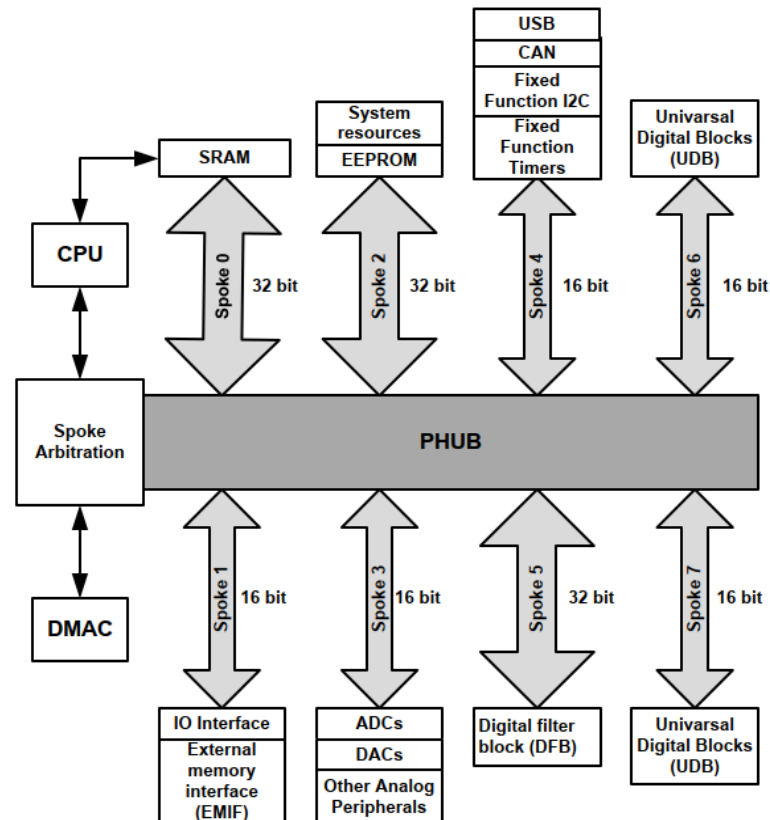
Hier setzt **DMA (Direct Memory Access)** an: Ein DMA-Controller übernimmt die Übertragung der Daten direkt zwischen Peripherie und Speicher, ohne dass die CPU jeden Schritt steuern muss. Die CPU konfiguriert den DMA-Controller einmalig und ist danach von der eigentlichen Datenbewegung entlastet. Wenn der DMA-Controller Zugriff auf den Systembus benötigt, fordert er diesen bei der CPU an (*Bus Request*). Über eine *Bus Grant*-Leitung teilt die CPU dem DMAC mit, dass er zugreifen darf. So werden Konflikte auf dem Bus vermieden (**Arbitrierung**). Währenddessen kann die CPU ggf. mit Daten aus dem Cache weiterarbeiten.

DMA spielt heute eine wichtige Rolle – z. B. beim Ausgeben von Audiodaten, beim Übertragen von Bilddaten in einen Videospeicher oder beim schnellen Kopieren von Speicherblöcken. Grundsätzlich unterstützt DMA den Datenaustausch zwischen:

Speicher ↔ Peripherie
Peripherie ↔ Peripherie
Speicher ↔ Speicher

Der PSoC5LP ermöglicht bis zu **24 unabhängig einstellbare DMA-Kanäle**. Speicher und Peripheriekomponenten sind über sogenannte **Spokes** an den **Peripherie-Hub (PHUB)** angebunden. Insgesamt gibt es acht solcher Spokes, sodass bis zu acht Transfers gleichzeitig ausgeführt werden können.

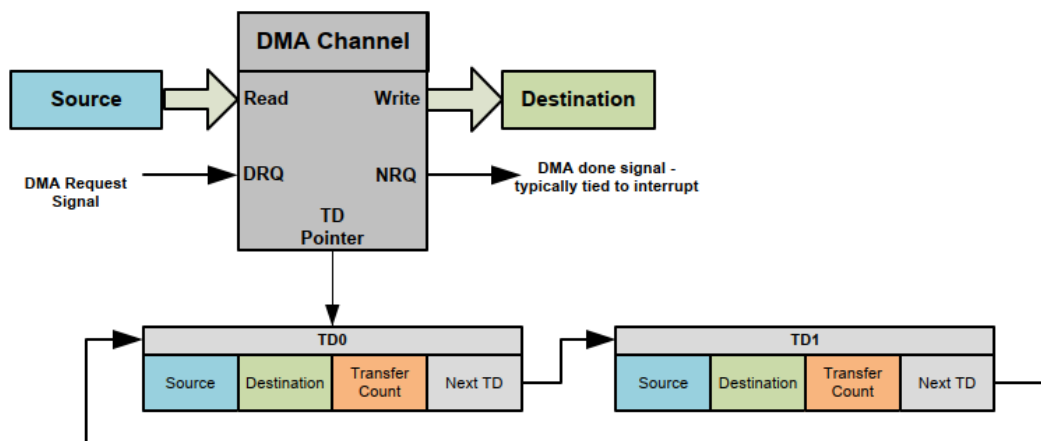
- **Spokes** haben eine Datenbreite von 16 oder 32 Bit. Angeschlossene Peripherien können 8-, 16- oder 32-Bit breit sein. Der PHUB passt die Übertragung automatisch an.
- Der PHUB hat zwei Bus-Master: **CPU** und **DMA-Controller**. Beide können gleichzeitig auf unterschiedliche Spokes zugreifen. Greifen sie auf denselben Spoke zu, entscheidet eine **Arbitrierung** über den Zugriff.



Bildquelle: AN52705 von Infineon

Jeder DMA-Transfer wird über einen **Transaction Descriptor (TD)** beschrieben.
Ein TD enthält:

- Quell- und Zieladresse,
- Anzahl der zu übertragenden Daten,
- Konfigurationsbits (z. B. Auto-Inkrement).



Bildquelle: AN52705 von Infineon

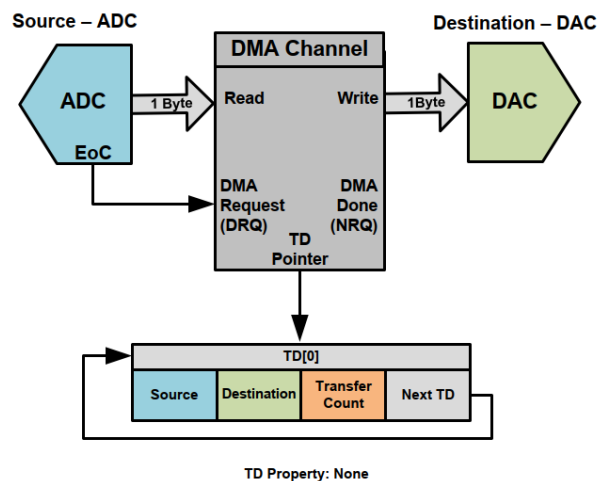
TDs können **verkettet** werden, um komplexe Transfers ohne CPU-Eingriff auszuführen. Jeder der 24 DMA-Kanäle kann eine **TD-Kette** abarbeiten (bis zu 128 TDs pro Kanal).

i Info: Man kann sich einen TD wie einen kleinen „Transferauftrag“ vorstellen, den der DMAC Schritt für Schritt abarbeitet.

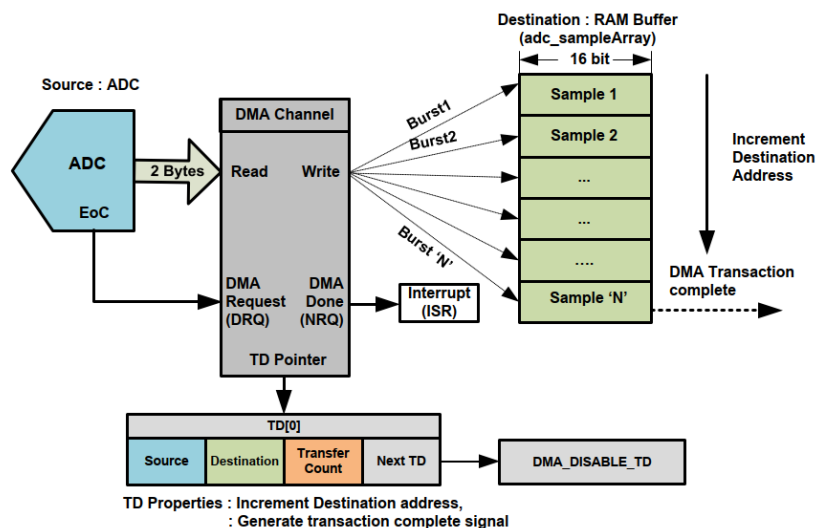
Ein DMA-Transfer kann von der CPU oder direkt von einer Peripherie ausgelöst werden (*DMA Request*).

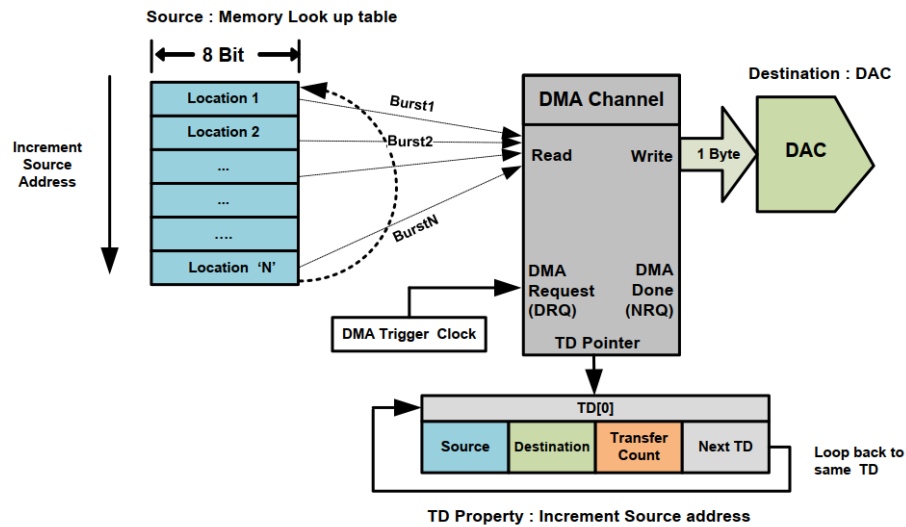
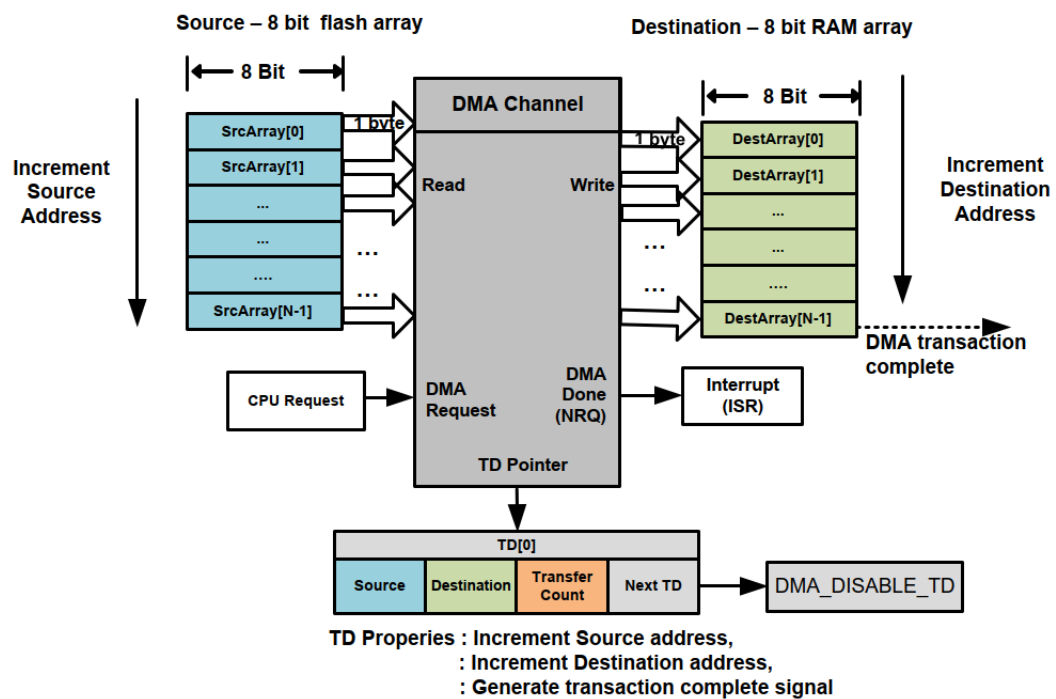
Im folgenden werden einige DMA-Beispielkonfigurationen vorgestellt (entnommen aus AN52705; dort findet man teilweise auch den Sourcecode zu den einzelnen Beispielen).

Peripherie → Peripherie:

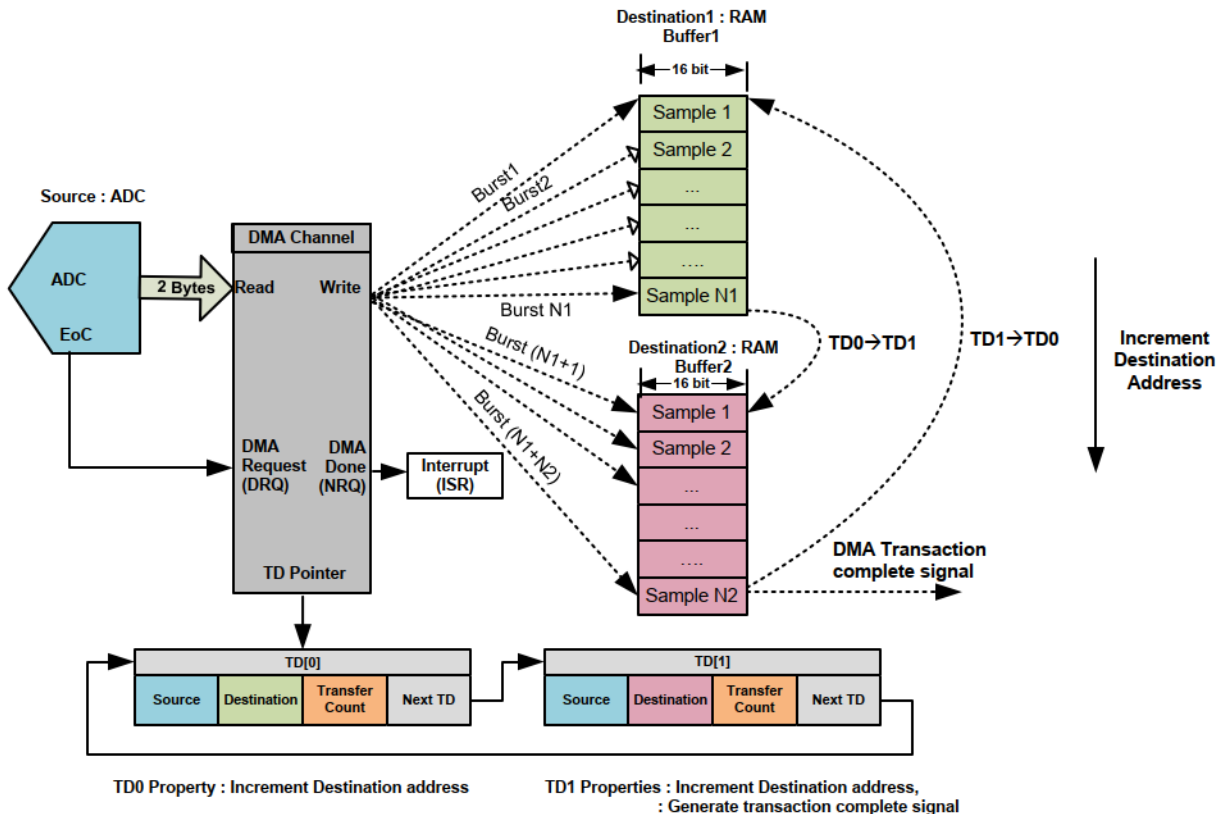


Peripherie → Speicher:



Speicher → Peripherie:Speicher → Speicher:

Peripherie → Speicher mit TD-Chaining:



Neben dem Transaction Descriptor (TD) muss auch der **DMA-Kanal** konfiguriert werden. Wichtige Parameter sind:

- **Quell- und Zieladresse,**
- **Anzahl der zu übertragenden Daten,**
- **Burst-Größe** (wie viele Bytes pro Transfer, bevor der Bus wieder freigegeben wird),
- **Verkettung mit weiteren TDs** (Next TD).

Die Konfiguration erfolgt komfortabel über den **DMA-Wizard** in PSoC Creator. Hier können auch optionale Signale berücksichtigt werden:

- **nrq (Transfer Complete):** signalisiert das Ende eines Transfers (z. B. an eine Interrupt-Komponente),
- **drq (Hardware Request):** startet einen DMA-Transfer (z. B. über *end-of-conversion* eines ADCs),
- **trq (Hardware Termination):** bricht eine laufende Übertragung ab.

Der Wizard übernimmt die detaillierte Registerkonfiguration und stellt sicher, dass die gewählten Parameter korrekt umgesetzt werden.

💡 Merke: DMA-Kanal vs. Spoke

- Ein **DMA-Kanal** beschreibt und steuert einen Datentransfer (Quelle, Ziel, Länge, Optionen).
- Ein **Spoke** ist die physikalische Bus-Anbindung im PHUB, über die Daten tatsächlich transportiert werden.

Mehrere DMA-Kanäle können über denselben Spoke laufen – es entscheidet die **Arbitrierung**, wer wann

den Bus nutzen darf.

**KI-Aufgabe: DMA in der WaveDAC-Komponente**

Bitte eine KI, dir zu erklären, wie die **WaveDAC8-Komponente** im PSoC5LP funktioniert und welche Rolle **DMA** dabei spielt. Achte darauf, dass du die Grundidee in **eigenen Worten** festhältst:

- Welche Aufgabe übernimmt der DAC?
- Warum werden zwei DMA-Kanäle verwendet?
- Wie entlastet DMA hier die CPU?

Ergänze deine Erklärung, indem du dir von der KI ein **Blockschaltbild** erstellen lässt, das den internen Aufbau der WaveDAC-Komponente veranschaulicht (Speicher ↔ DMA-Kanäle ↔ DAC ↔ Ausgang). Nutze dieses Bild, um die Funktionsweise der Komponente im Gespräch mit deinem Banknachbarn zu erklären.